

模式识别与计算机视觉

图像的基本操作

张振宇

南京大学智能科学与技术学院

2025

Image Filtering

- Types of image transformations.
- Point image processing.
- Linear shift-invariant image filtering.
- Convolution.
- Image gradients.

Types of image transformations

What is an image?



What is an image?



A (color) image
is a 3D tensor
of numbers.

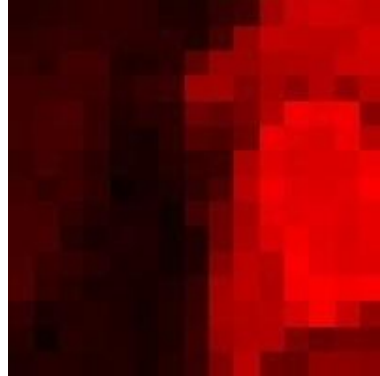
What is an image?



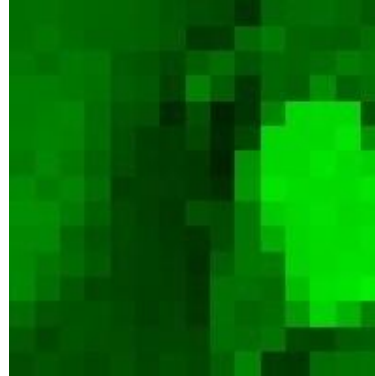
color image patch

How many bits are
the intensity values?

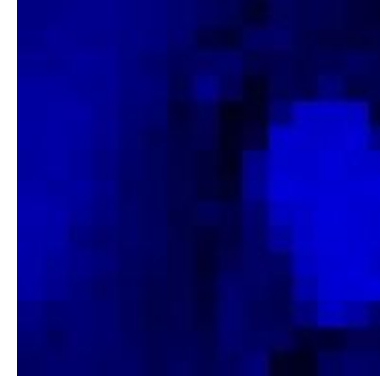
red



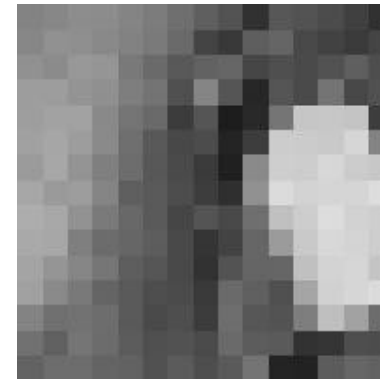
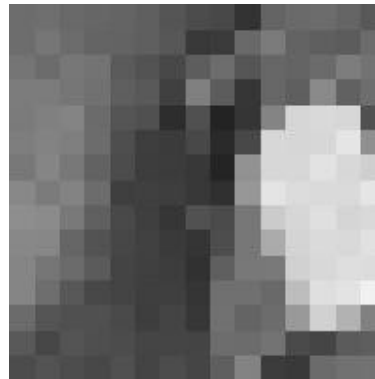
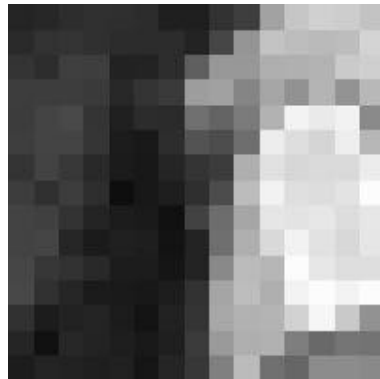
green



blue



colorized for visualization



actual intensity values per channel

Each channel
is a 2D array of
numbers.

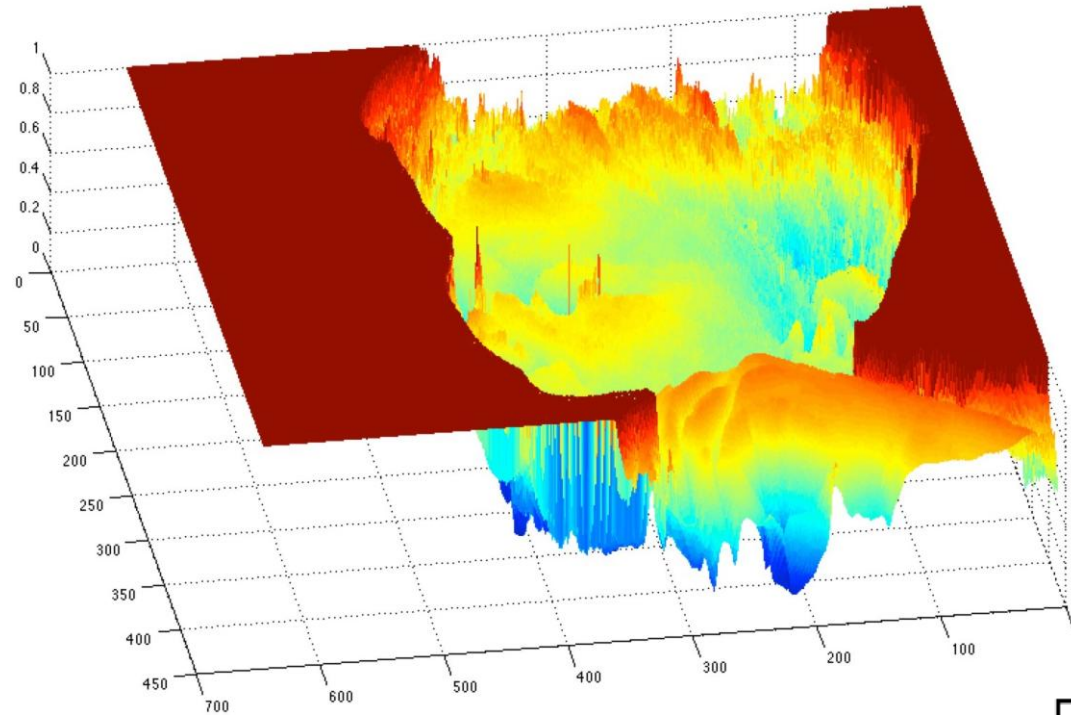
What is an image?



grayscale image

What is the range of the image function f ?

$$f(\mathbf{x})$$



domain $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

A (grayscale) image is a 2D function.

What types of image transformations can we do?



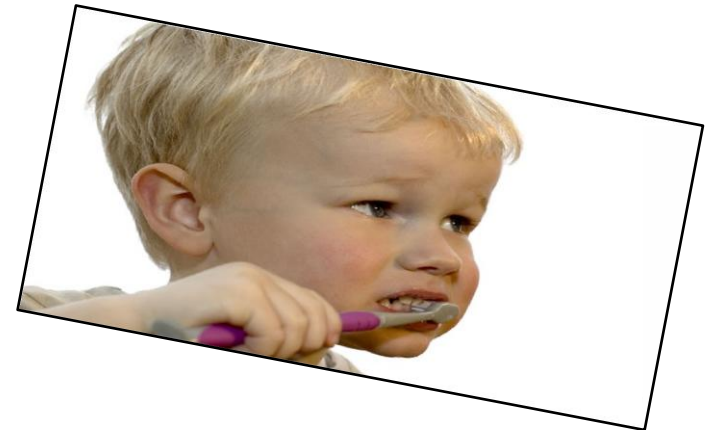
Filtering



changes pixel *values*



Warping



changes pixel *locations*

What types of image transformations can we do?

F



Filtering



$$G(\mathbf{x}) = h\{F(\mathbf{x})\}$$

G



changes *range* of image function

F

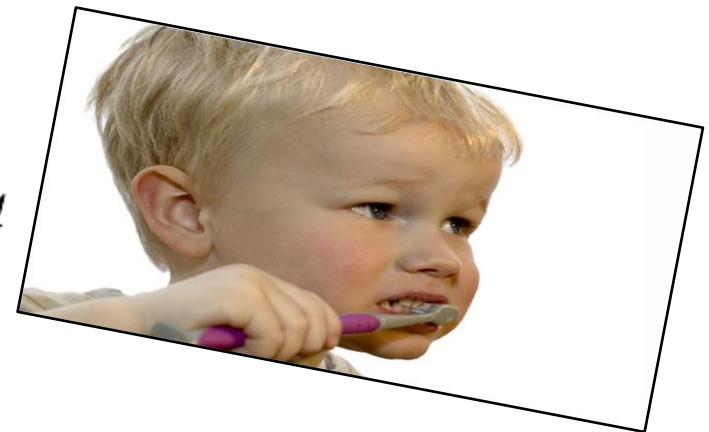


Warping



$$G(\mathbf{x}) = F(h\{\mathbf{x}\})$$

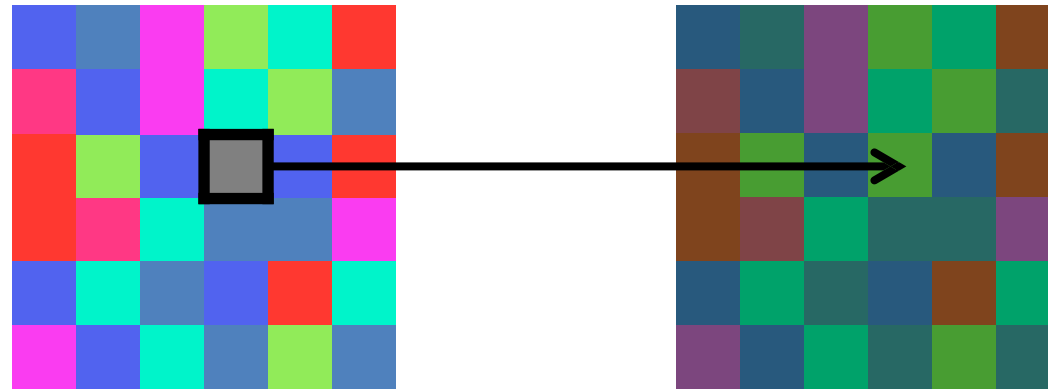
G



changes *domain* of image function

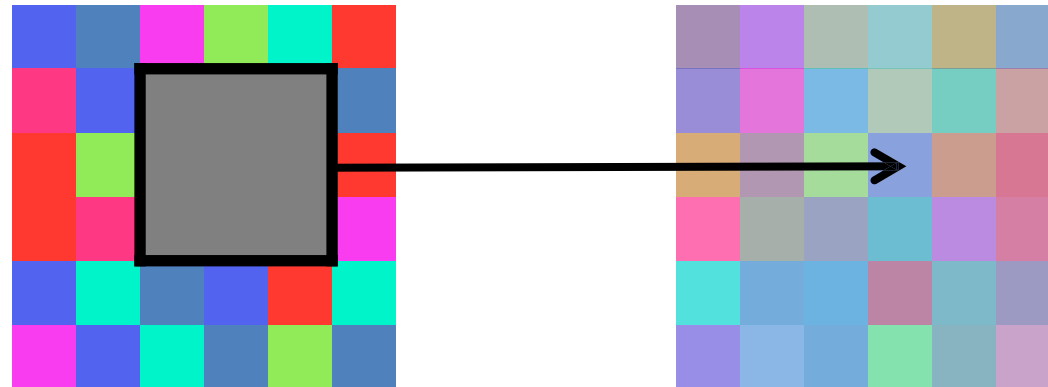
What types of image filtering can we do?

Point Operation



point processing

Neighborhood Operation



“filtering”

Point processing

Examples of point processing

original



darken



lower contrast



non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



x

darken



lower contrast



non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



x

darken



$x - 128$

lower contrast



non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

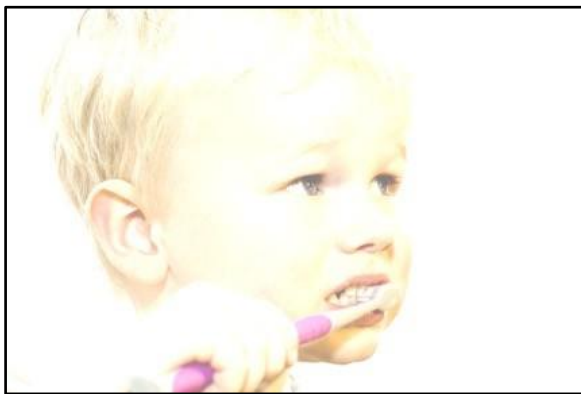
non-linear lower contrast



invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

non-linear lower contrast



$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

non-linear lower contrast



$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

non-linear lower contrast



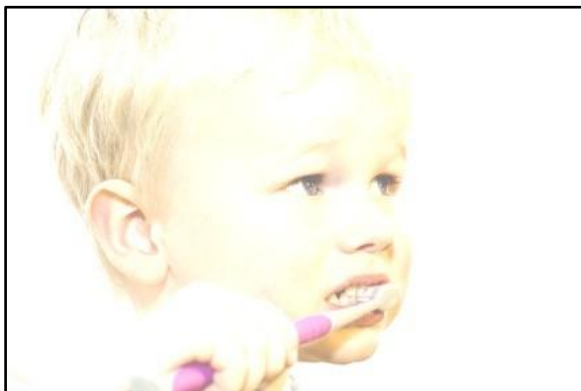
$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



$$x + 128$$

raise contrast



non-linear raise contrast



How would you
implement these?

Examples of point processing

original



$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

non-linear lower contrast



$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



$$x + 128$$

raise contrast



$$x \times 2$$

non-linear raise contrast



How would you
implement these?

Examples of point processing

original



$$x$$

darken



$$x - 128$$

lower contrast



$$\frac{x}{2}$$

non-linear lower contrast



$$\left(\frac{x}{255}\right)^{1/3} \times 255$$

invert



$$255 - x$$

lighten



$$x + 128$$

raise contrast



$$x \times 2$$

non-linear raise contrast



$$\left(\frac{x}{255}\right)^2 \times 255$$

Many other types of point processing

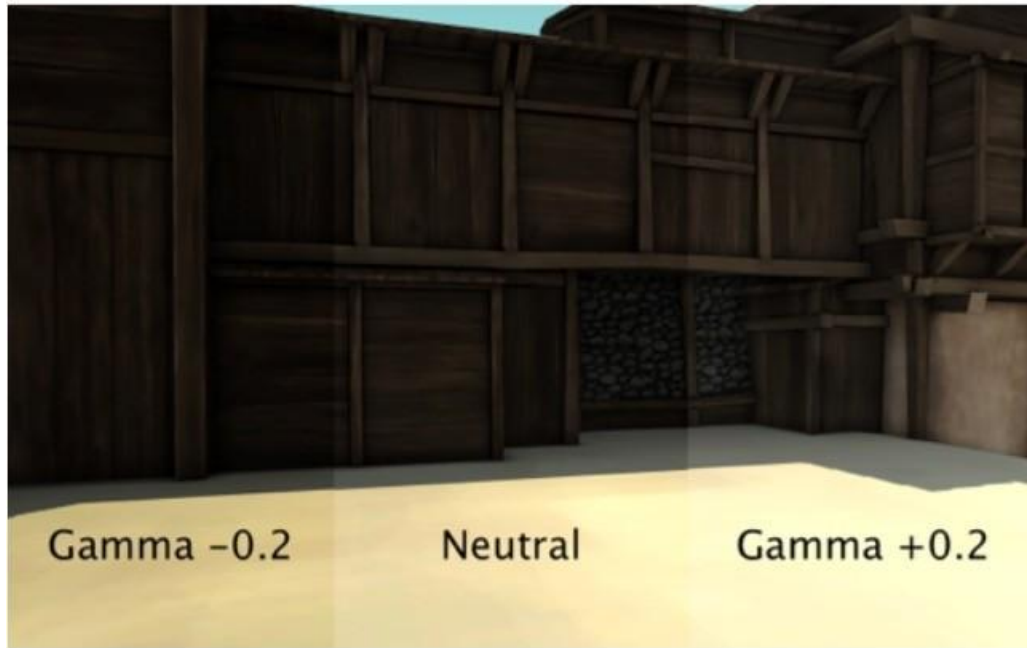


camera output



image after stylistic tonemapping

Many other types of point processing



Linear shift-invariant image filtering

Linear shift-invariant image filtering

- Replace each pixel by a *linear* combination of its neighbors (and possibly itself).
- The combination is determined by the filter's *kernel*.
- The same kernel is *shifted* to all pixel locations so that all pixels use the same linear combination of their neighbors.

Example: the box filter

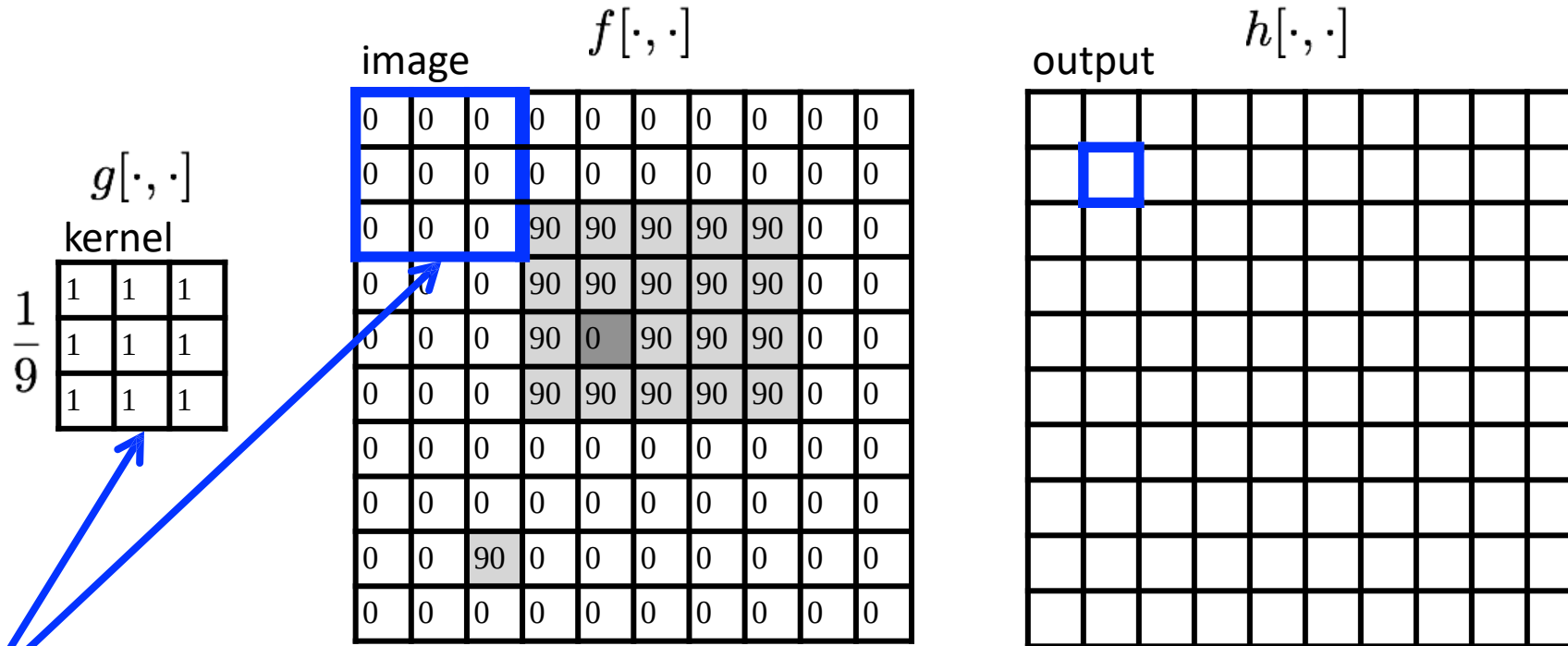
- also known as the 2D rect (not rekt) filter
- also known as the square mean filter

$$\text{kernel } g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- replaces pixel with local average
- has smoothing (blurring) effect



Let's run the box filter

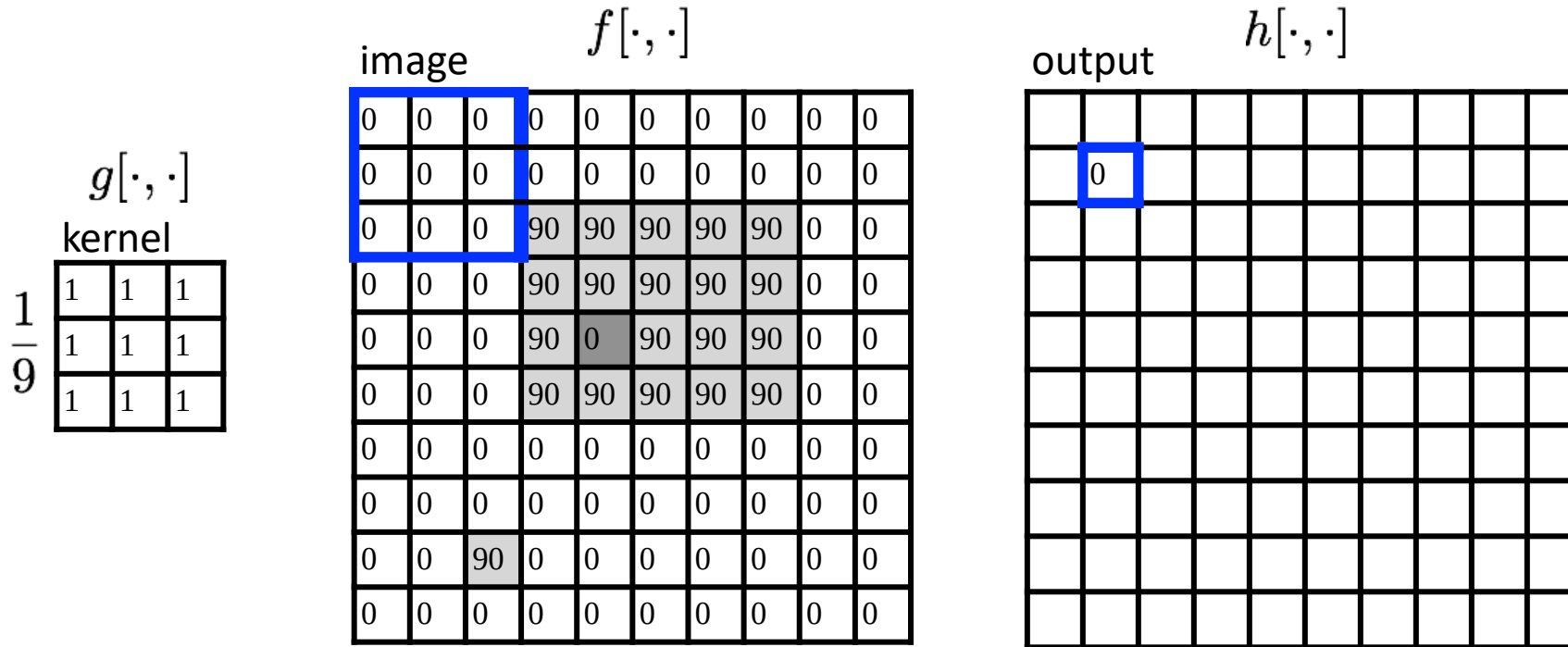


note that we assume that
the kernel coordinates
are centered

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
filter
image (signal)

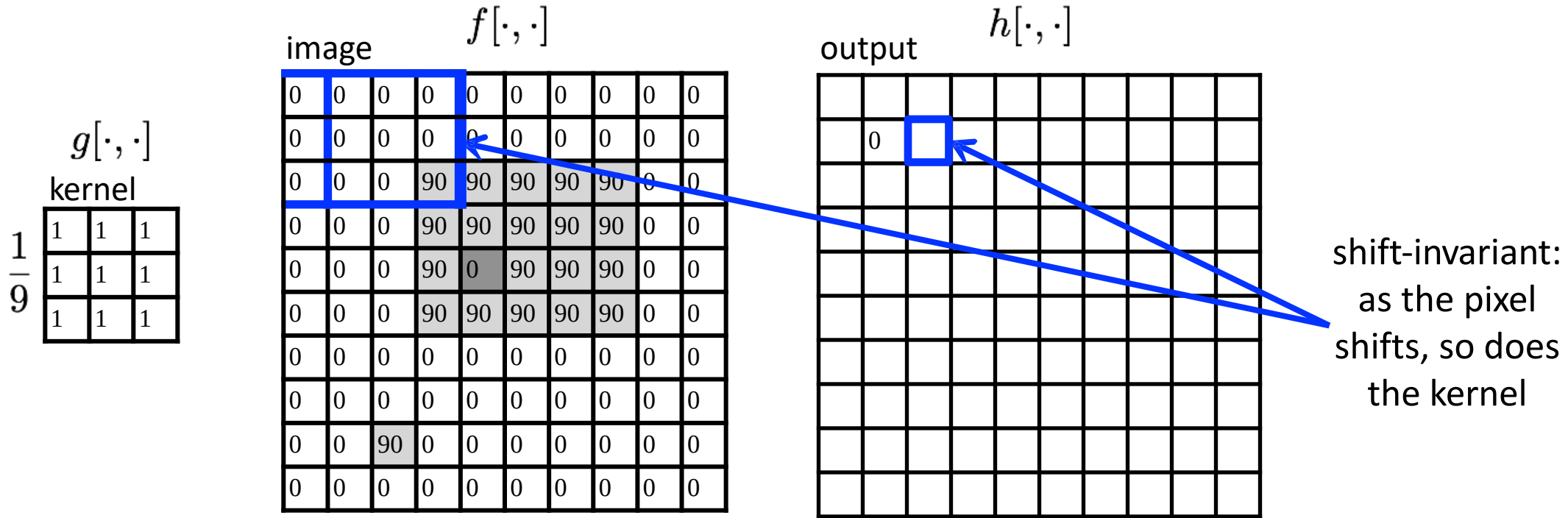
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

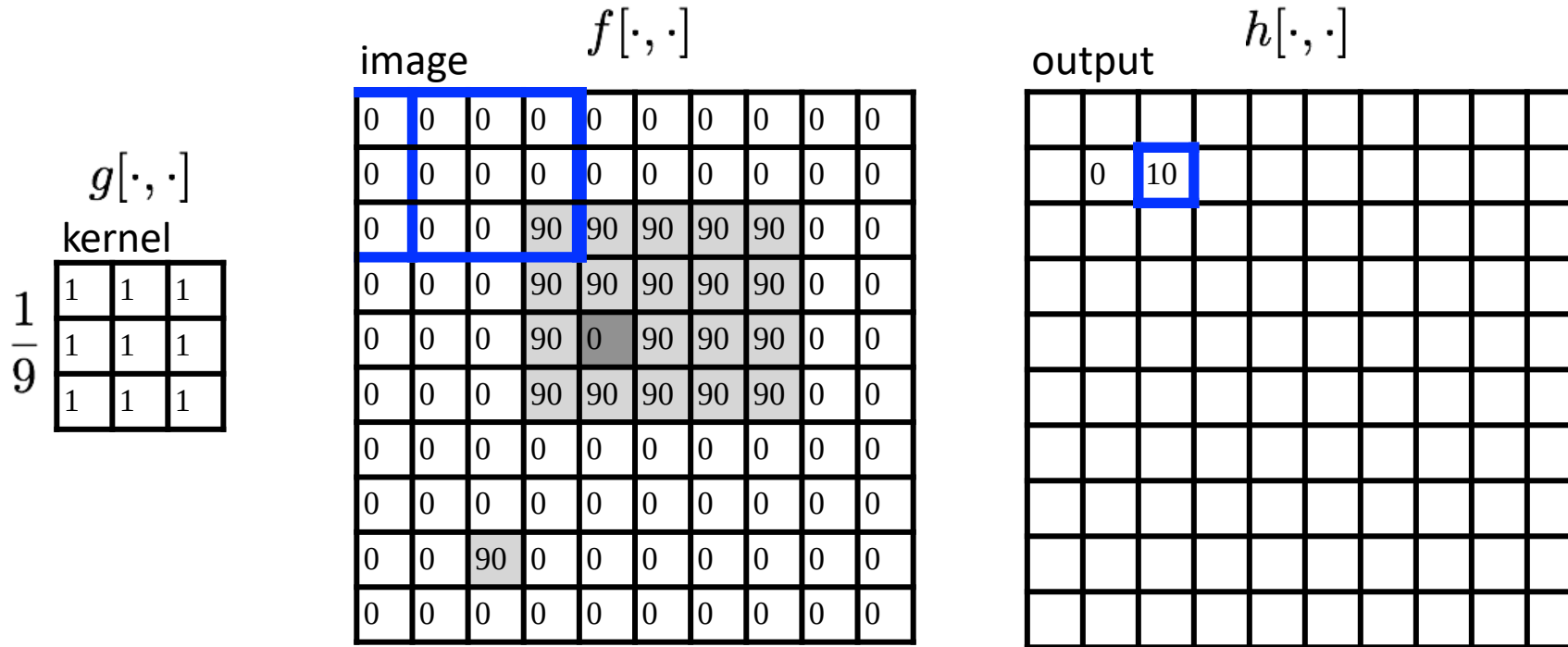
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output filter image (signal)

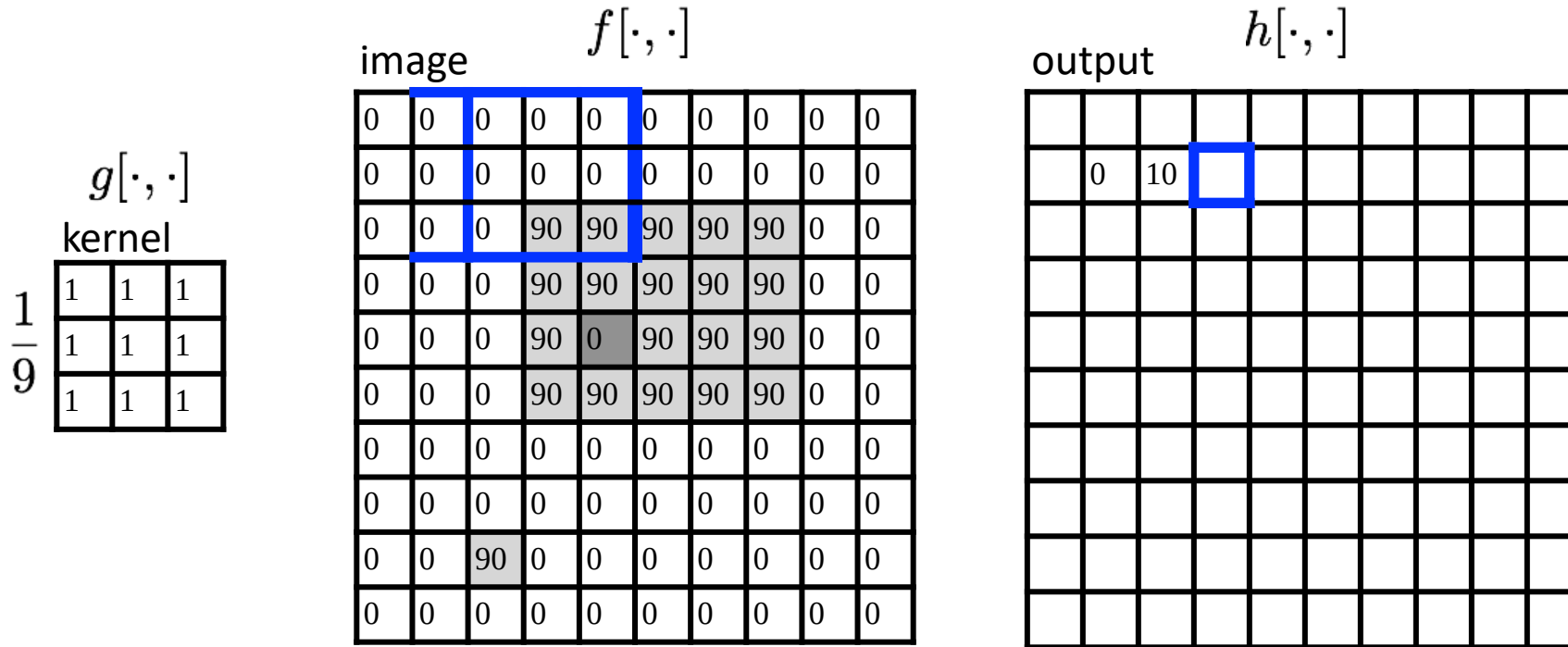
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

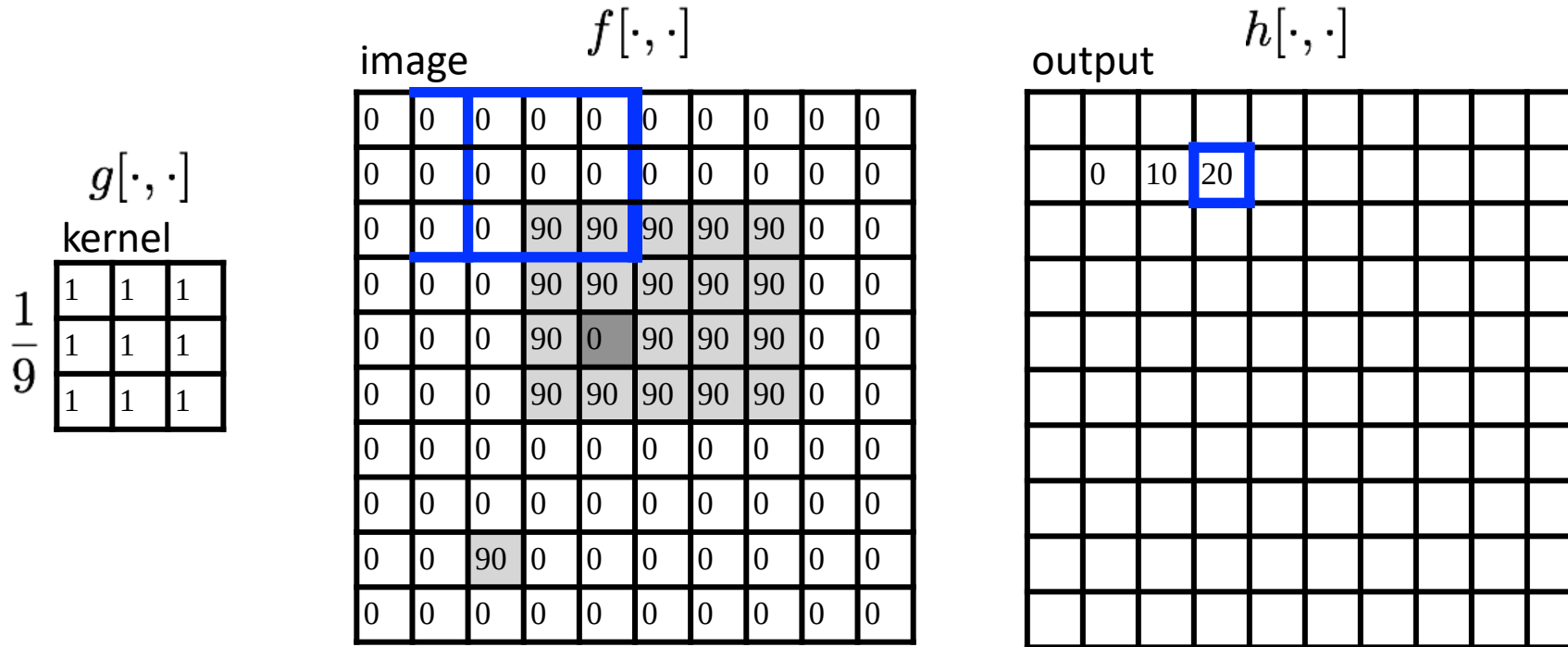
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

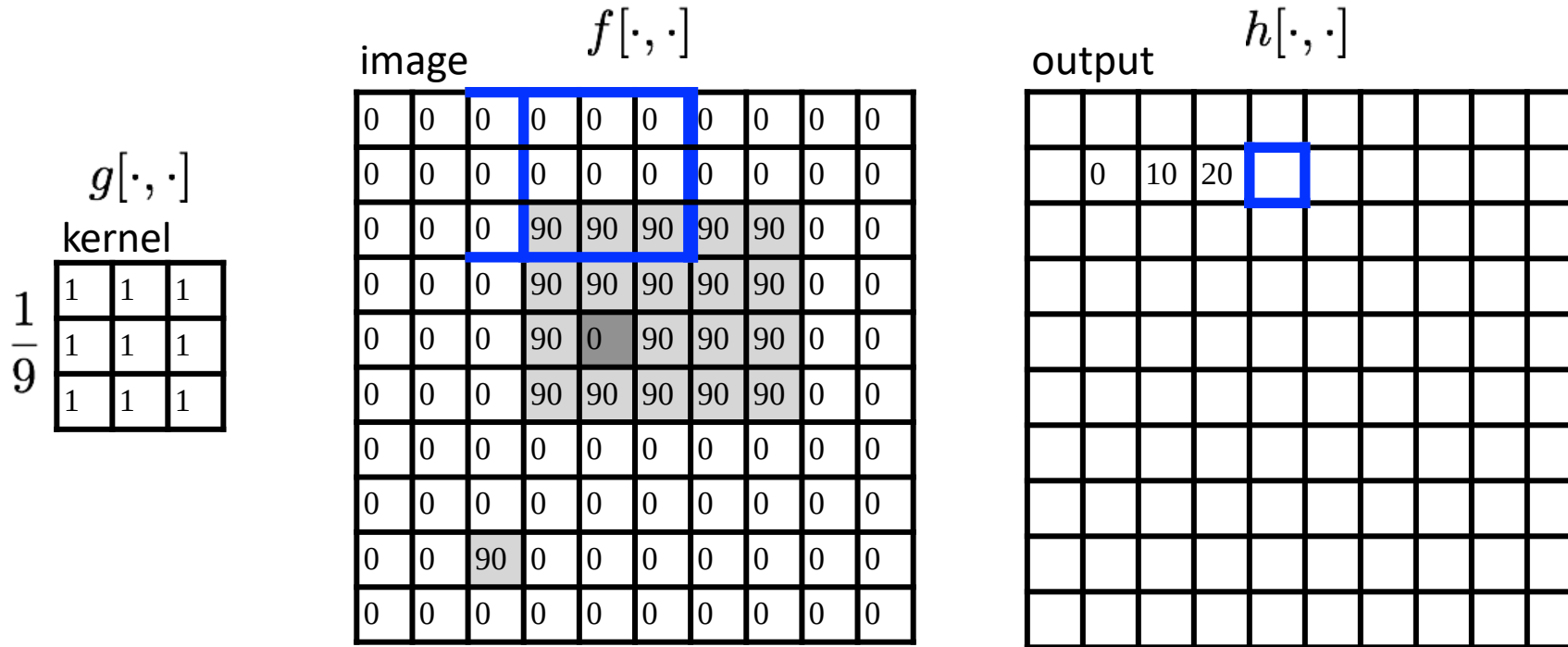
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

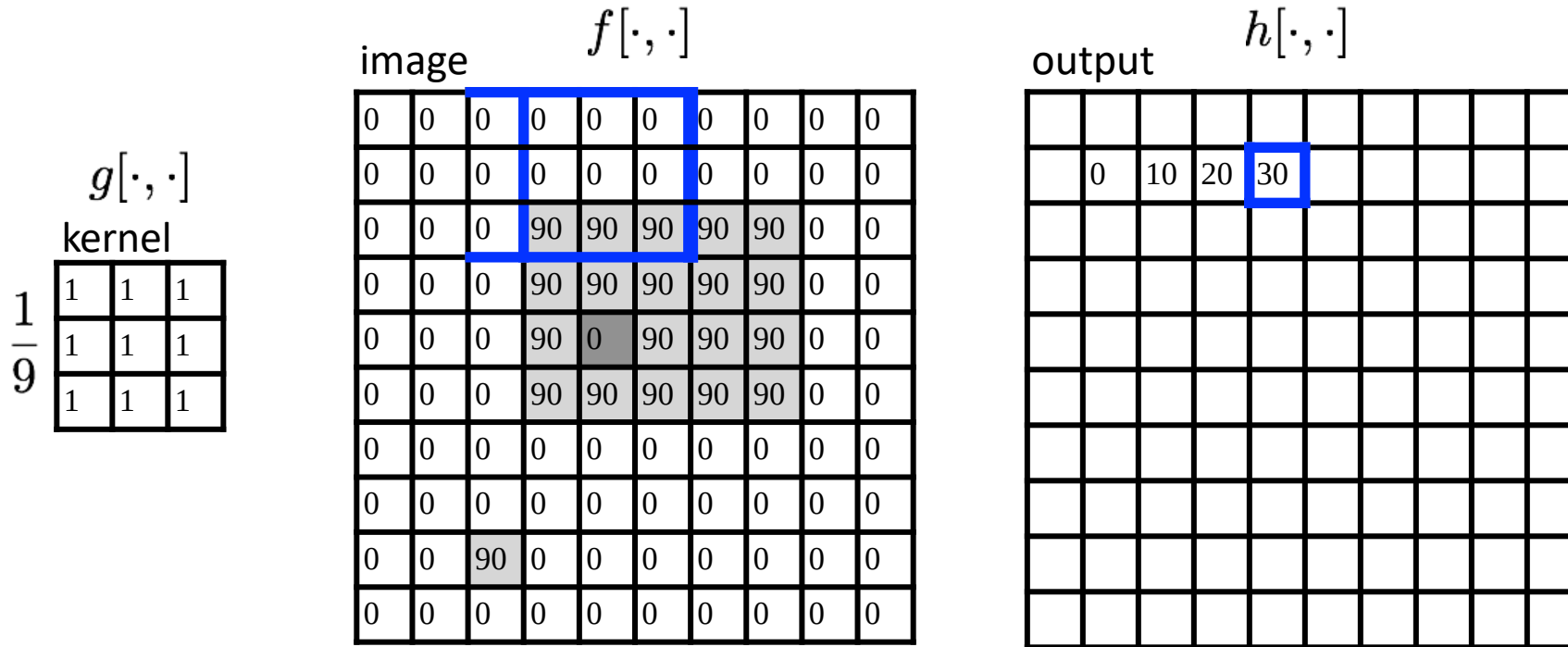
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

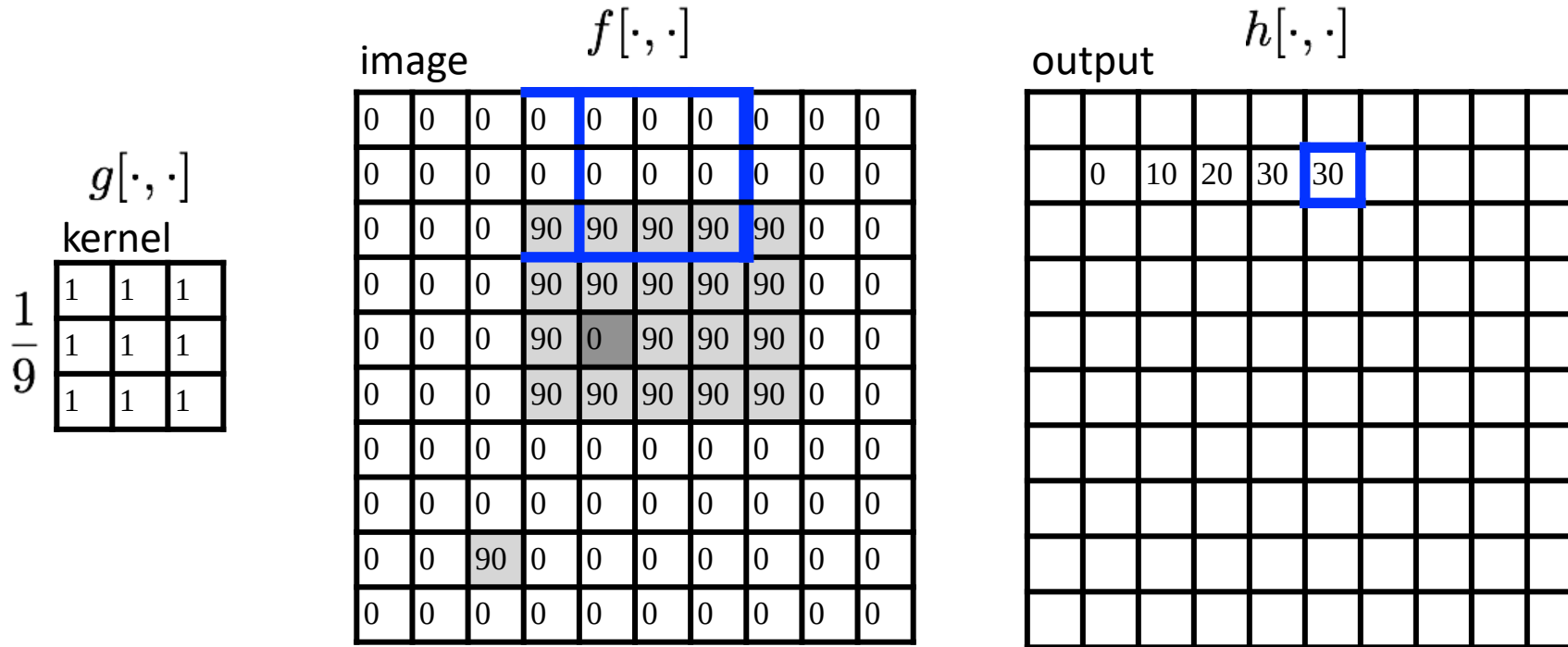
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

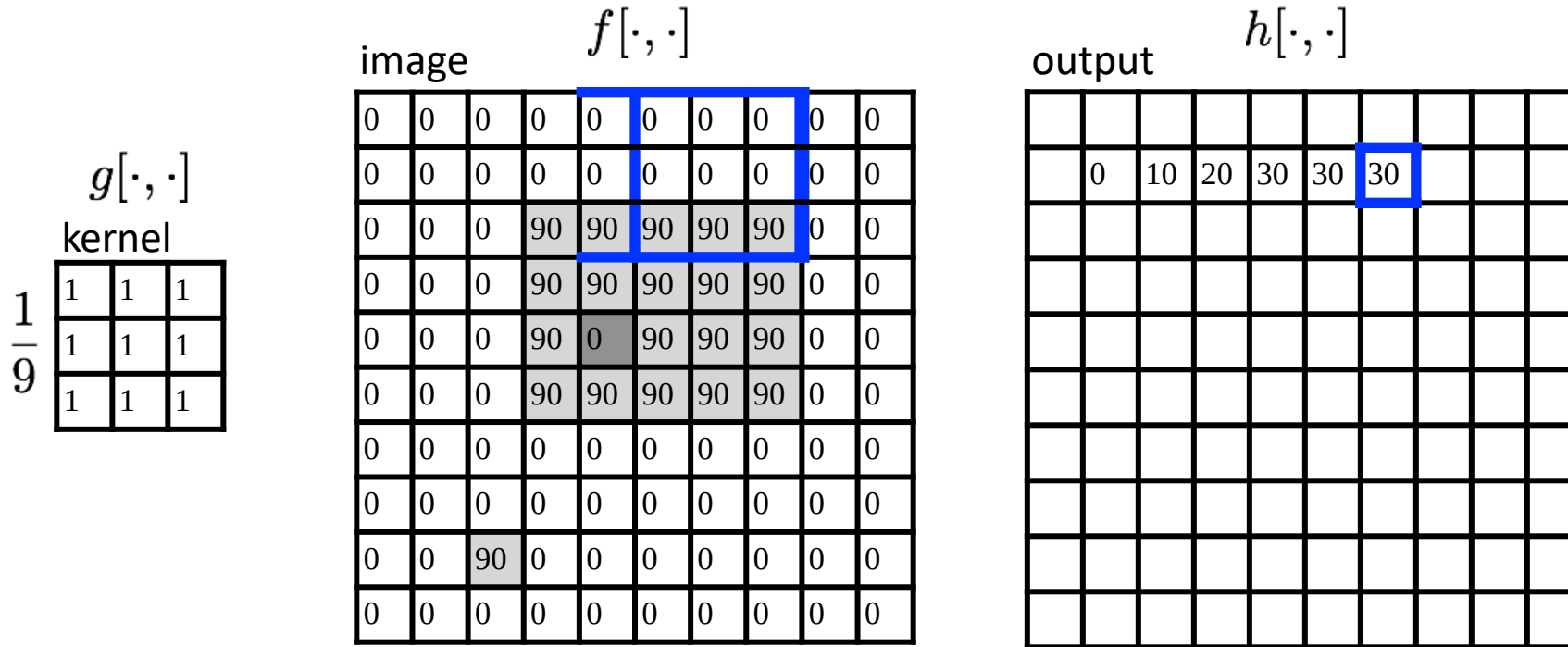
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

Let's run the box filter

$$\frac{1}{9} \begin{matrix} g[\cdot, \cdot] \\ \text{kernel} \\ \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20		

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} \begin{matrix} g[\cdot, \cdot] \\ \text{kernel} \\ \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

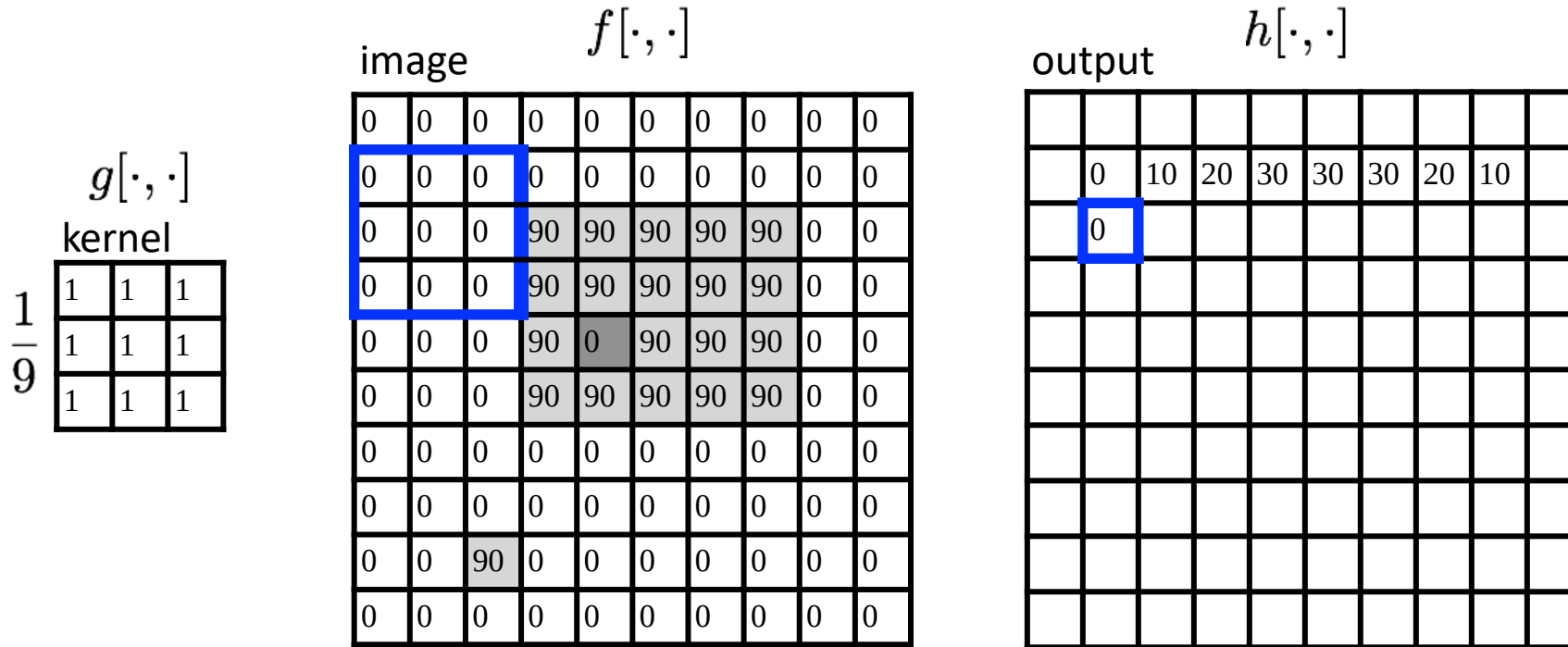
output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

kernel

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

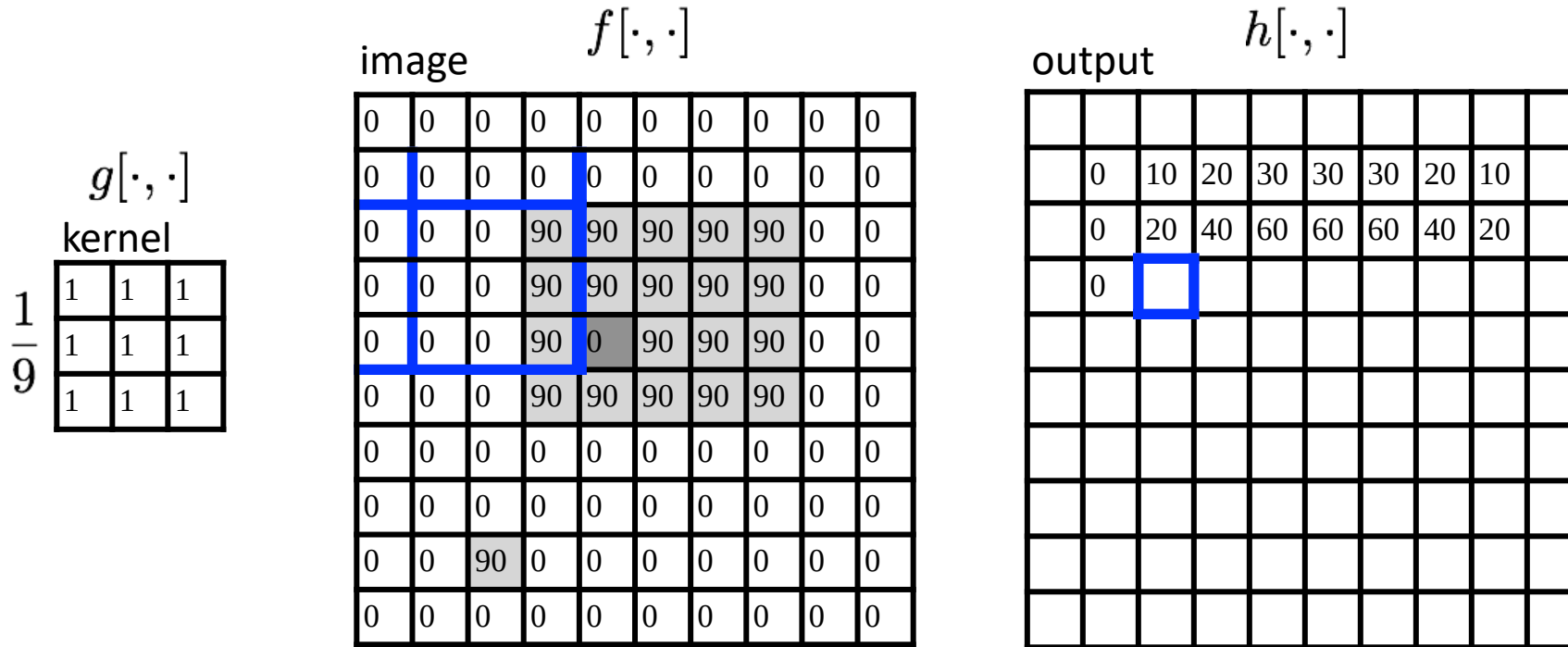
output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40						

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

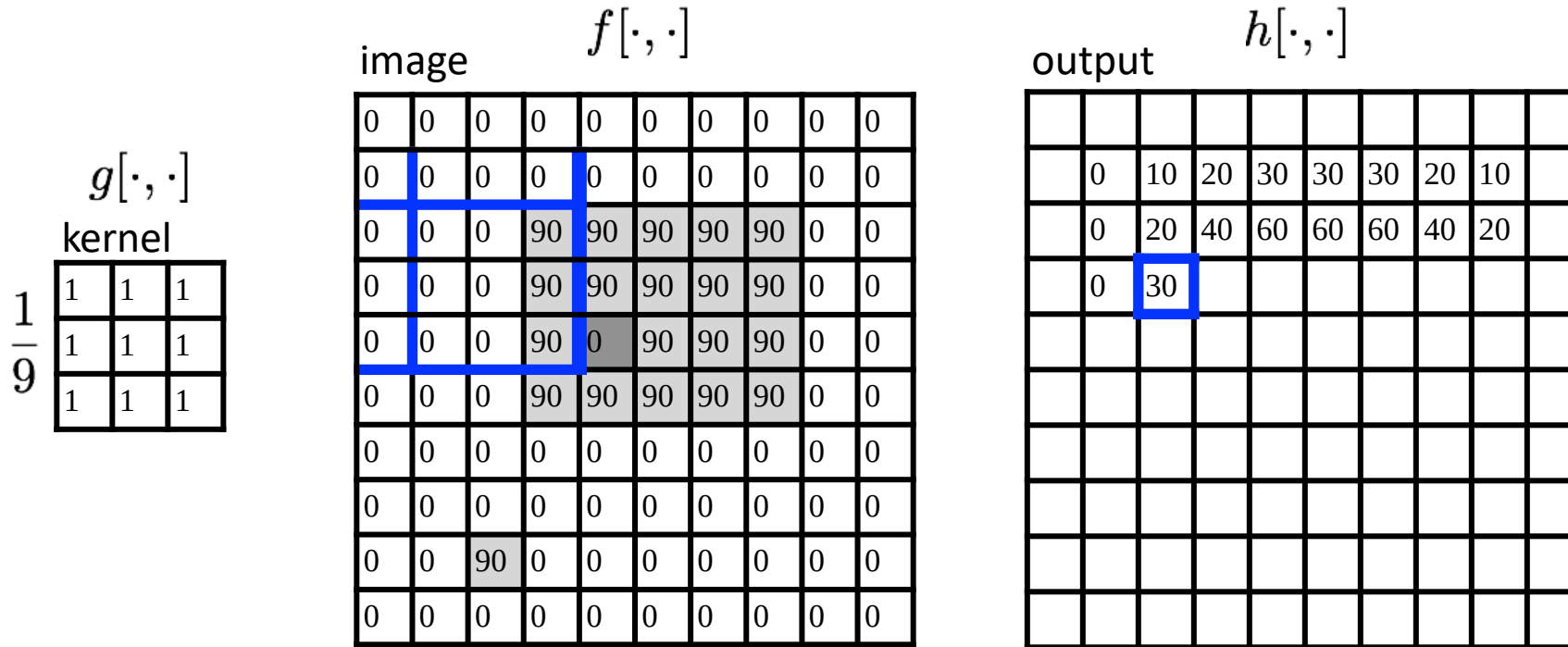
Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output k, l filter image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

kernel

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10								

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

Let's run the box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

kernel

image $f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

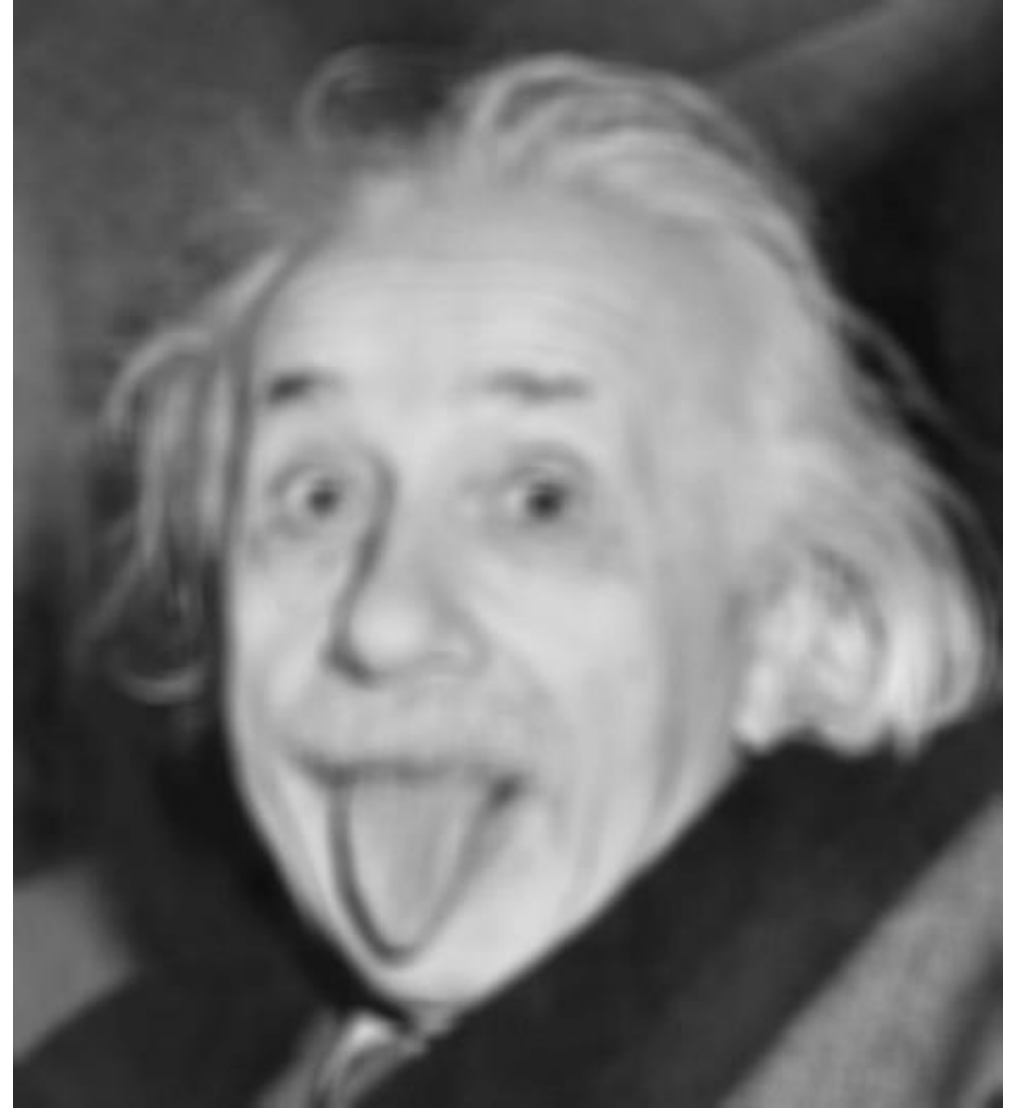
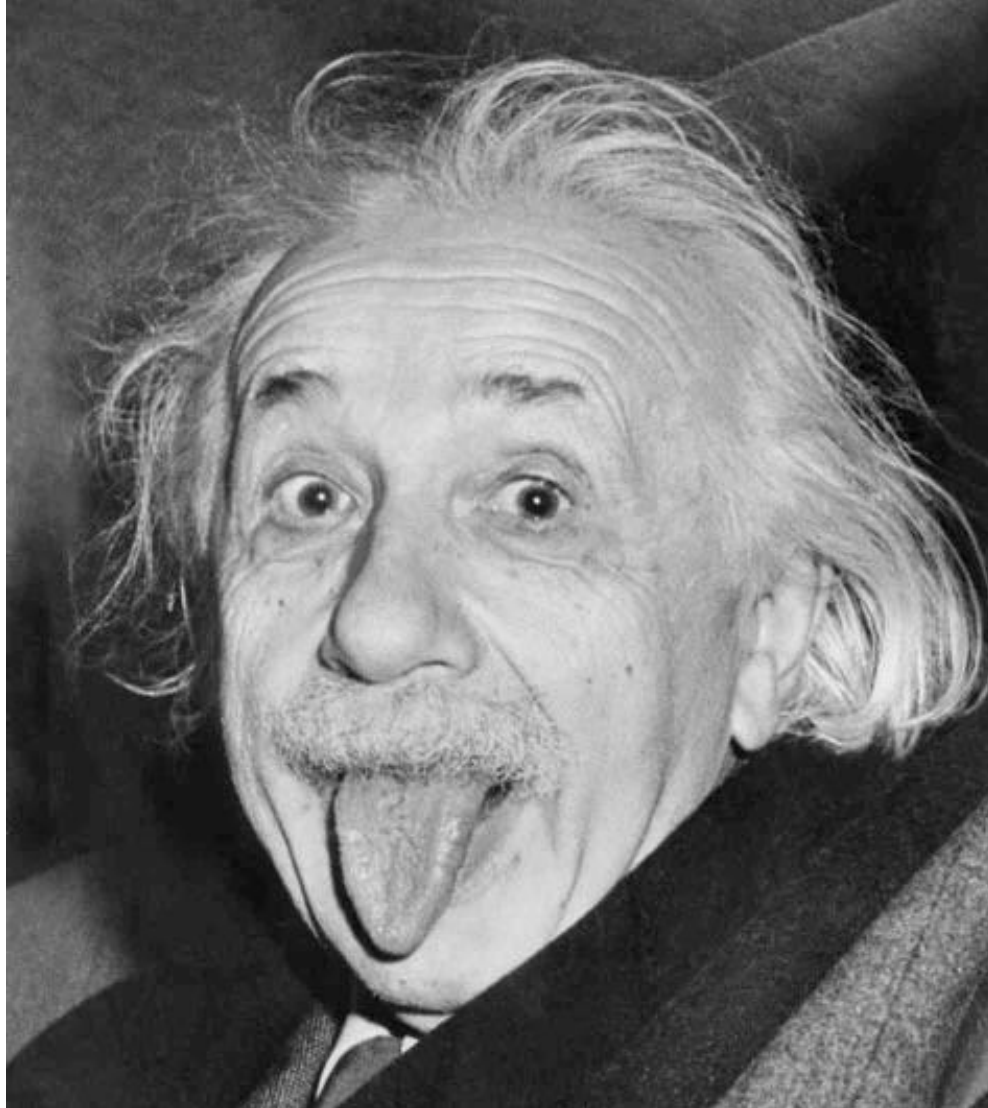
output $h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10	10	10	10	0	0	0	0	

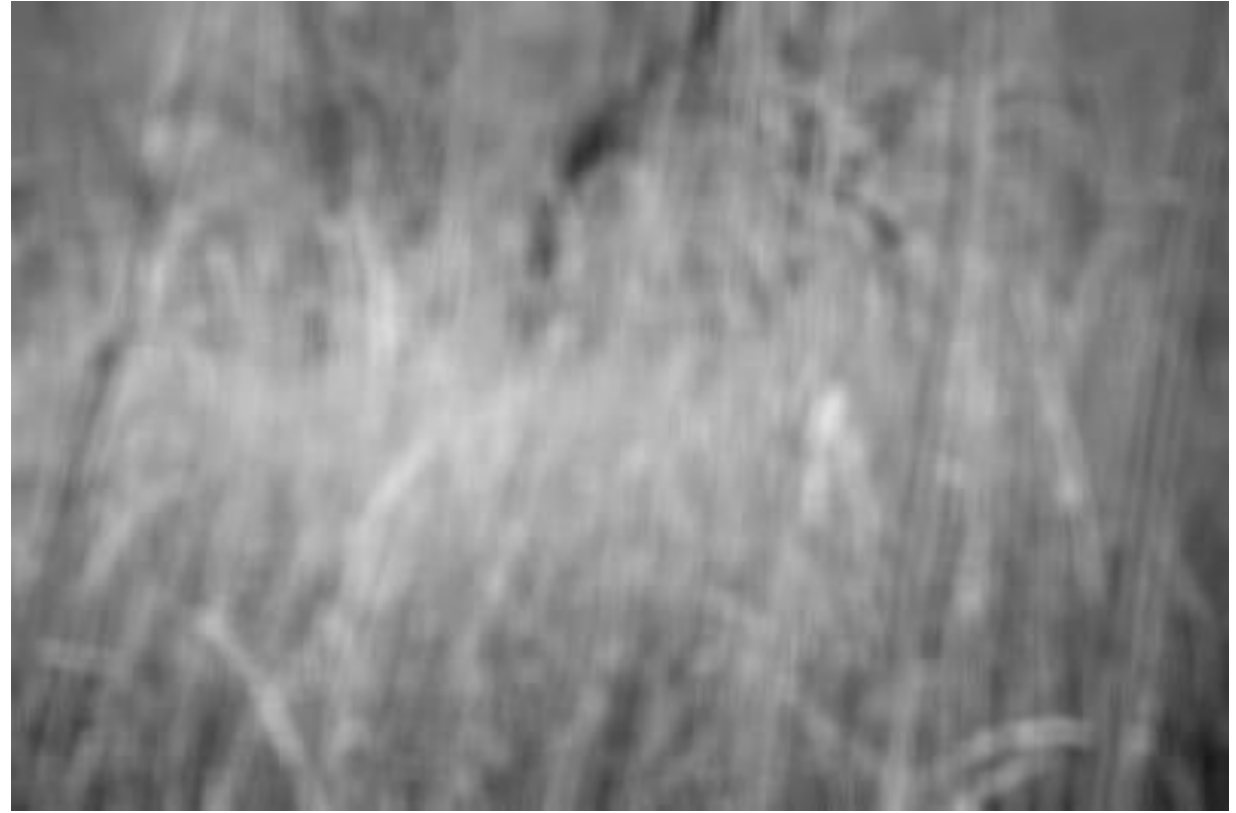
$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 k, l
filter
image (signal)

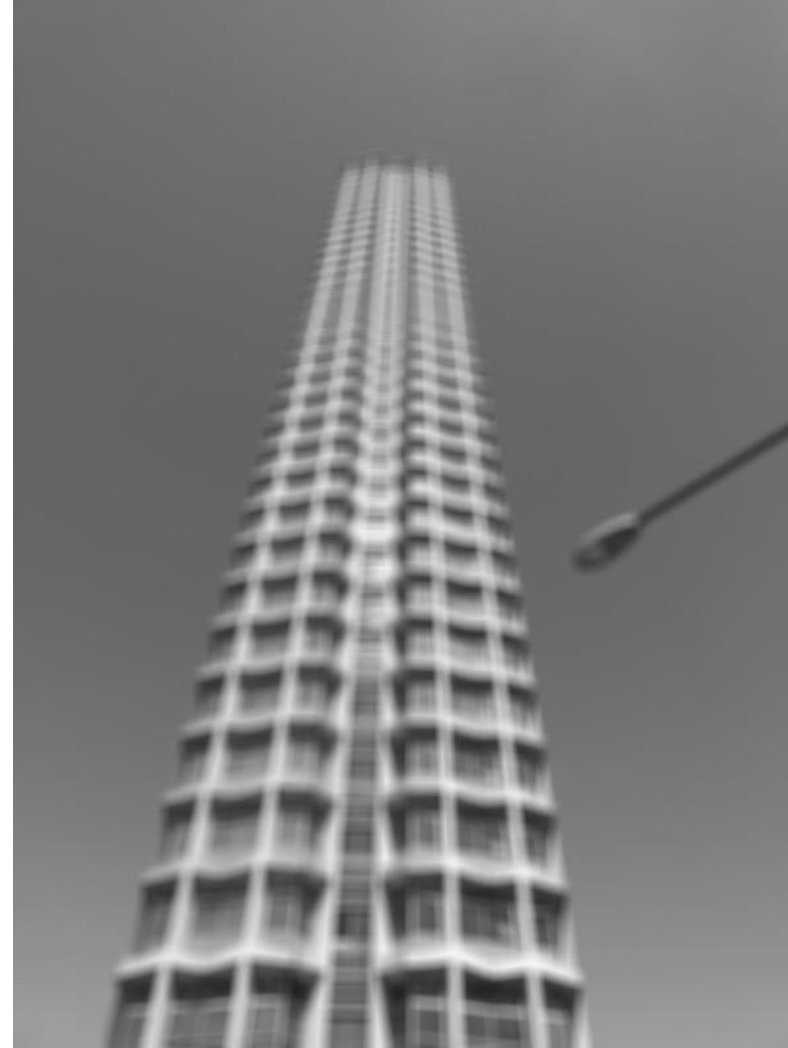
Some more realistic examples



Some more realistic examples



Some more realistic examples



Convolution

Convolution for 2D discrete signals

Definition of filtering as convolution:

$$(f * I)(x, y) = \sum_{i, j = -\infty}^{\infty} f(i, j) I(x - i, y - j)$$

Diagram illustrating the convolution operation with annotations:

- filtered image: points to $(f * I)(x, y)$
- filter: points to $f(i, j)$
- input image: points to $I(x - i, y - j)$
- notice the flip: points to the minus signs in $x - i$ and $y - j$

If the filter $f(i, j)$ is non-zero only within $-1 \leq i, j \leq 1$,
then

$$(f * I)(x, y) = \sum_{i, j = -1}^1 f(i, j) I(x - i, y - j)$$

The kernel we saw earlier is the 3x3 matrix representation of $f(i, j)$.

Separable filters

A 2D filter is separable if it can be written as the product of a “column” and a “row”.

example:
box filter

1	1	1
1	1	1
1	1	1

=

1
1
1

column

*

1	1	1
---	---	---

row

What is the rank of this filter matrix?

Separable filters

A 2D filter is separable if it can be written as the product of a “column” and a “row”.

example:
box filter

1	1	1
1	1	1
1	1	1

=

1
1
1

column

*

1	1	1
---	---	---

row

Why is this important?

Separable filters

A 2D filter is separable if it can be written as the product of a “column” and a “row”.

example:
box filter

1	1	1
1	1	1
1	1	1

=

1
1
1

column

*

1	1	1
---	---	---

row

2D convolution with a separable filter is equivalent to two 1D convolutions (with the “column” and “row” filters).

Separable filters

A 2D filter is separable if it can be written as the product of a “column” and a “row”.

example:
box filter

1	1	1
1	1	1
1	1	1

=

1
1
1

column

*

1	1	1
---	---	---

row

2D convolution with a separable filter is equivalent to two 1D convolutions (with the “column” and “row” filters).

If the image has $M \times M$ pixels and the filter kernel has size $N \times N$:

- What is the cost of convolution with a non-separable filter?

Separable filters

A 2D filter is separable if it can be written as the product of a “column” and a “row”.

example:
box filter

1	1	1
1	1	1
1	1	1

=

1
1
1

column

*

1	1	1
---	---	---

row

2D convolution with a separable filter is equivalent to two 1D convolutions (with the “column” and “row” filters).

If the image has $M \times M$ pixels and the filter kernel has size $N \times N$:

- What is the cost of convolution with a non-separable filter? $\longrightarrow M^2 \times N^2$
- What is the cost of convolution with a separable filter?

Separable filters

A 2D filter is separable if it can be written as the product of a “column” and a “row”.

example:
box filter

1	1	1
1	1	1
1	1	1

=

1
1
1

column

*

1	1	1
---	---	---

row

2D convolution with a separable filter is equivalent to two 1D convolutions (with the “column” and “row” filters).

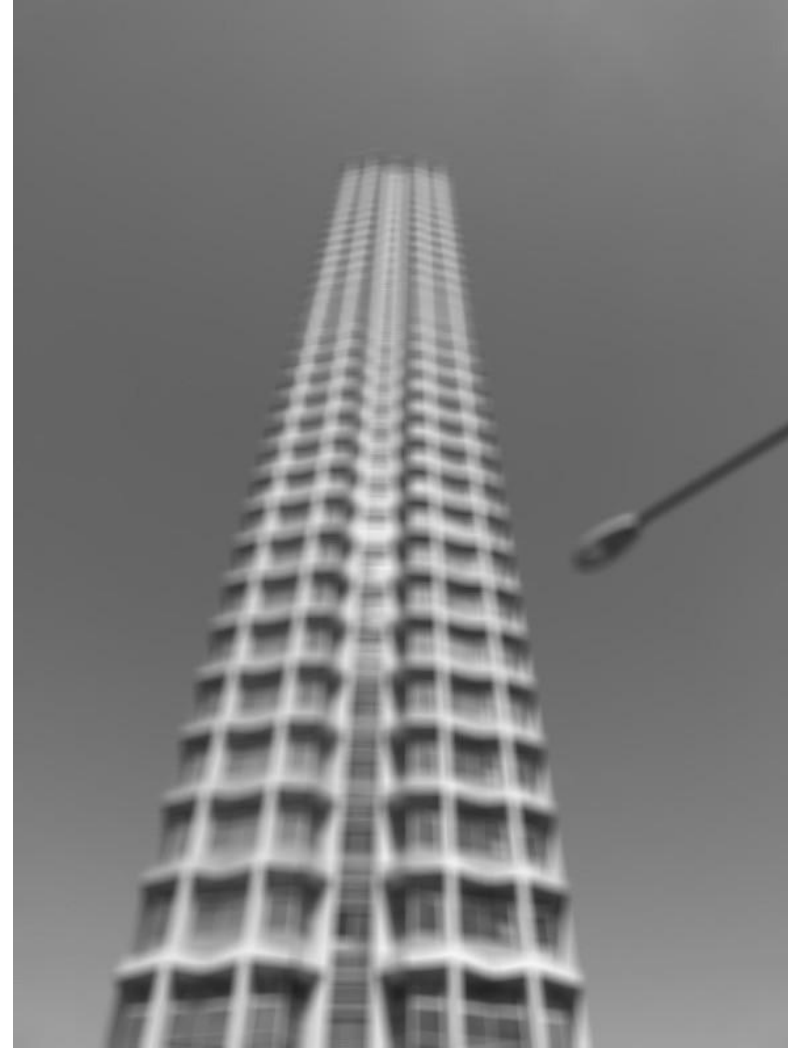
If the image has $M \times M$ pixels and the filter kernel has size $N \times N$:

- What is the cost of convolution with a non-separable filter? $\longrightarrow M^2 \times N^2$
- What is the cost of convolution with a separable filter? $\longrightarrow 2 \times N \times M^2$

A few more filters



original



3x3 box filter

do you see
any problems
in this image?

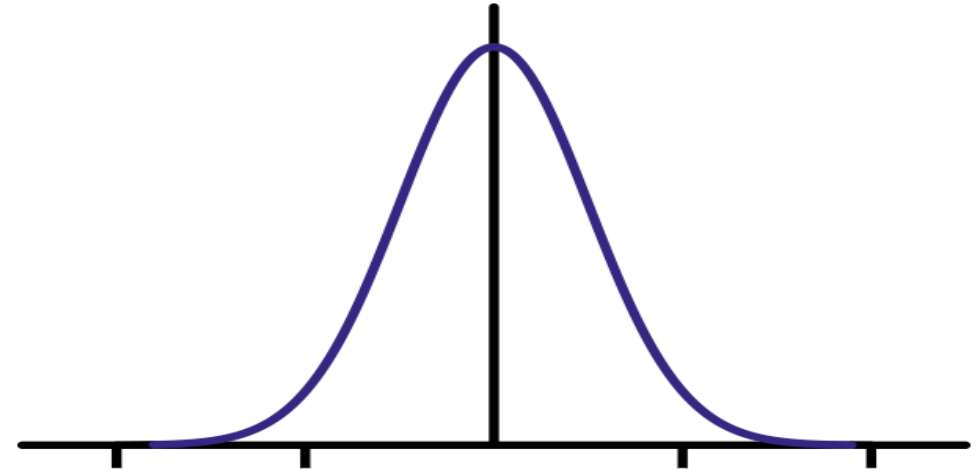
The Gaussian filter

- named (like many other things) after Carl Friedrich Gauss
- kernel values sampled from the 2D Gaussian function:

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

- weight falls off with distance from center pixel
- theoretically infinite, in practice truncated to some maximum distance

Any heuristics for selecting where to truncate?



The Gaussian filter

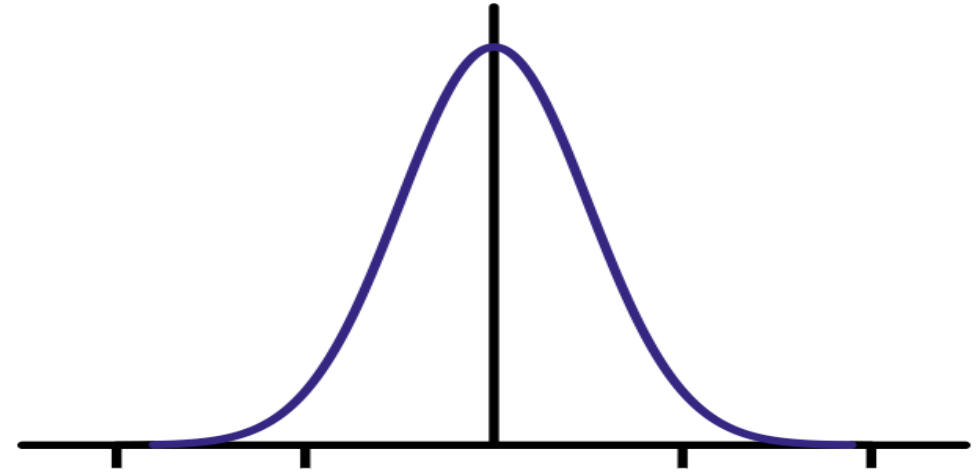
- named (like many other things) after Carl Friedrich Gauss
- kernel values sampled from the 2D Gaussian function:

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

- weight falls off with distance from center pixel
- theoretically infinite, in practice truncated to some maximum distance

Any heuristics for selecting where to truncate?

- usually at $2-3\sigma$



Is this a separable filter?

kernel $\frac{1}{16}$

1	2	1
2	4	2
1	2	1

The Gaussian filter

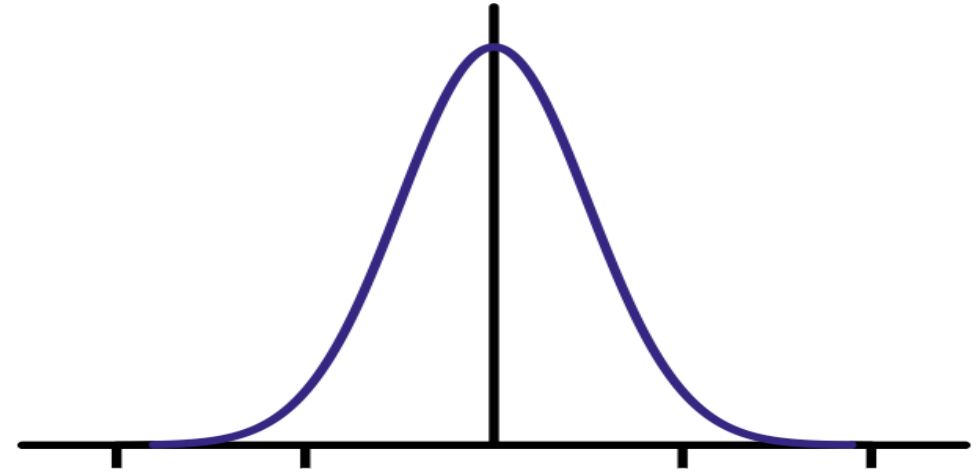
- named (like many other things) after Carl Friedrich Gauss
- kernel values sampled from the 2D Gaussian function:

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

- weight falls off with distance from center pixel
- theoretically infinite, in practice truncated to some maximum distance

Any heuristics for selecting where to truncate?

- usually at $2-3\sigma$

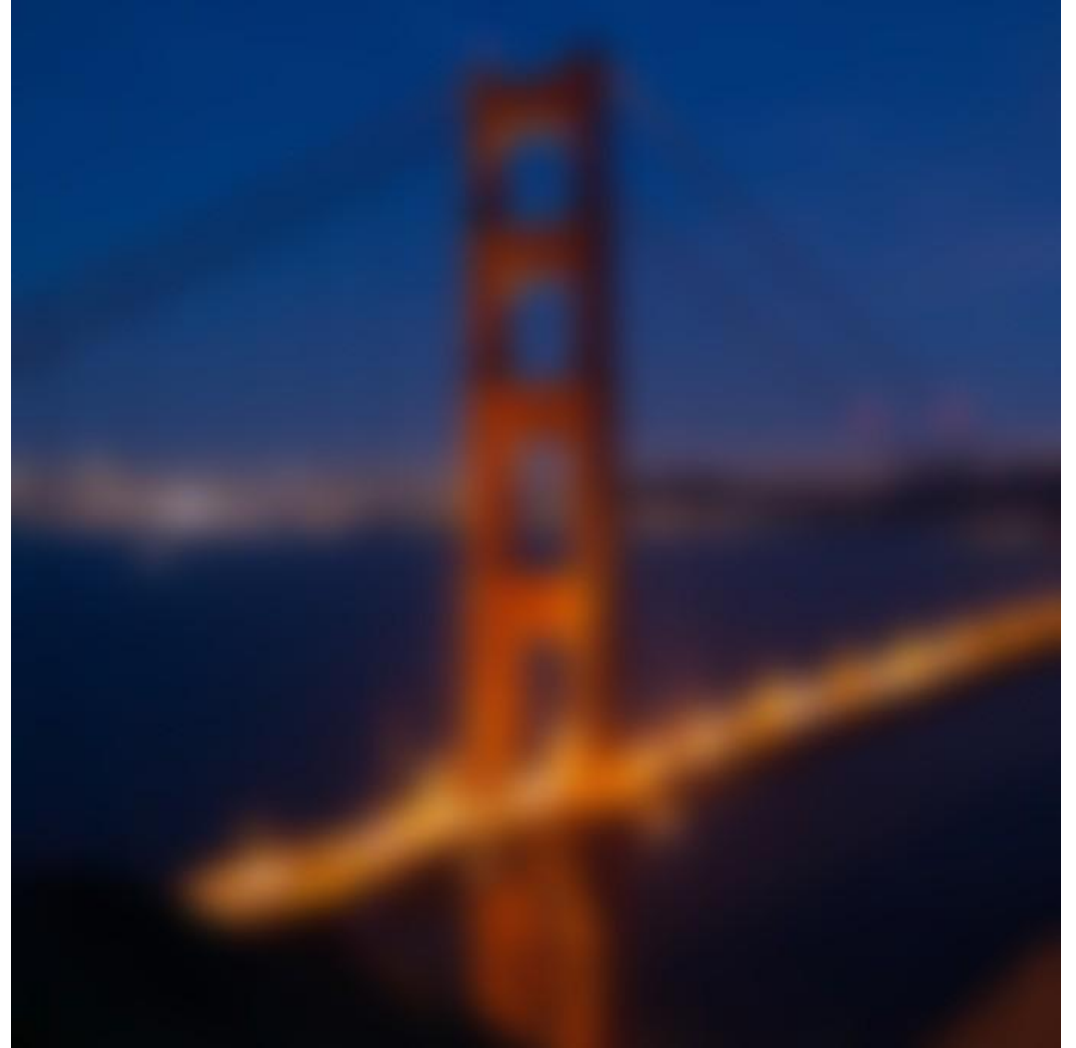


Is this a separable filter? Yes!

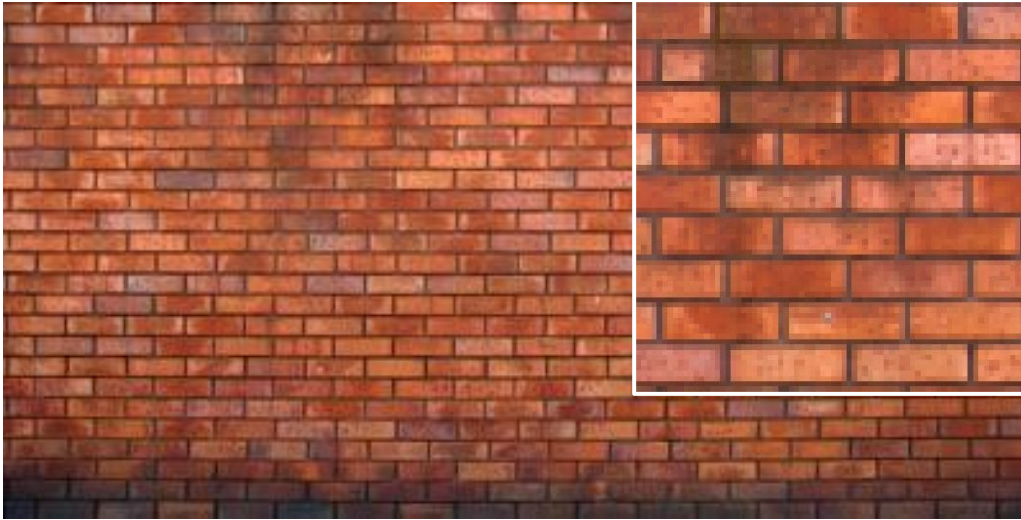
kernel $\frac{1}{16}$

1	2	1
2	4	2
1	2	1

Gaussian filtering example

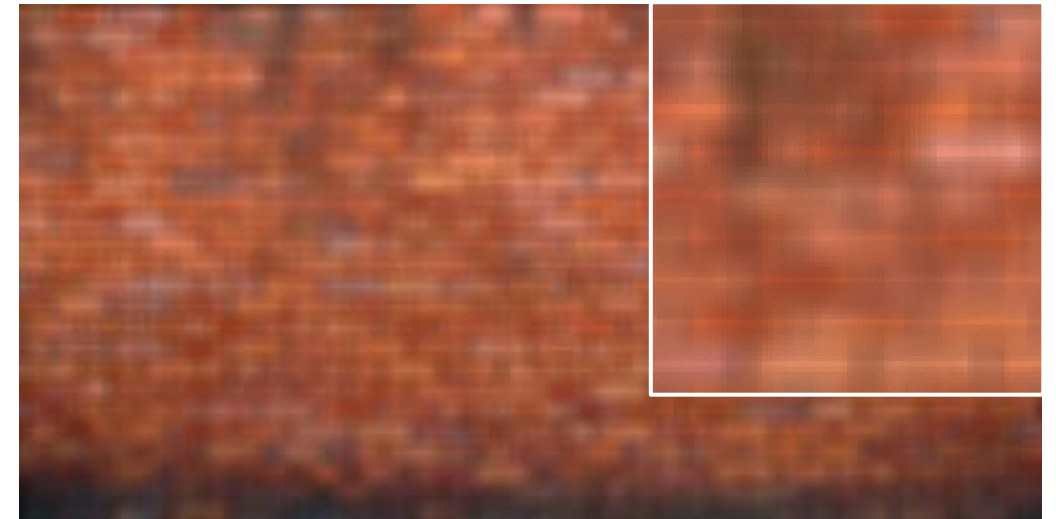
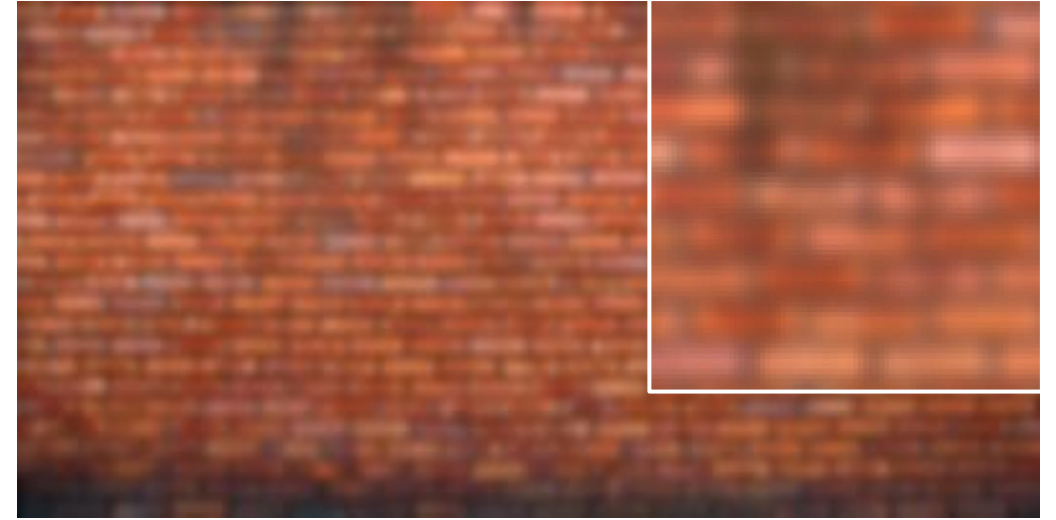


Gaussian vs box filtering

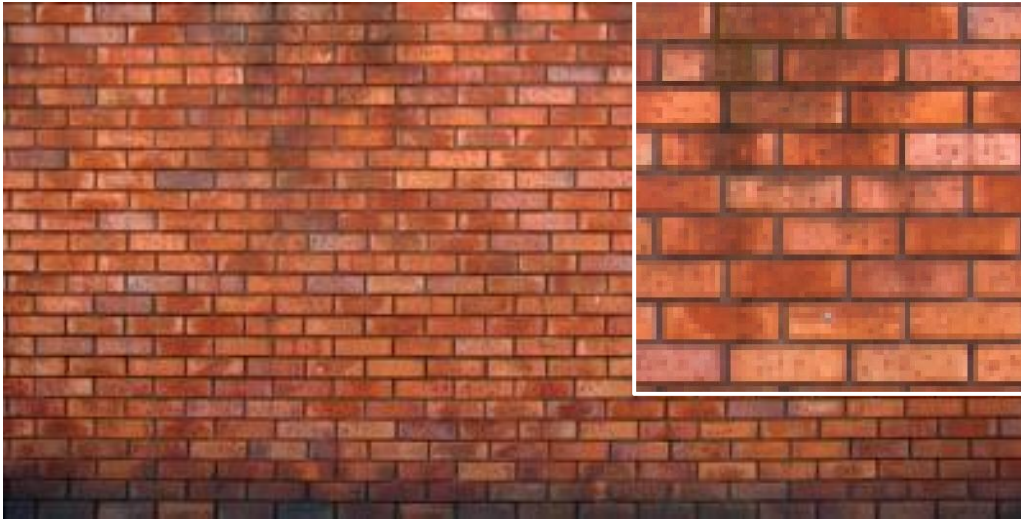


original

Which blur do you like better?

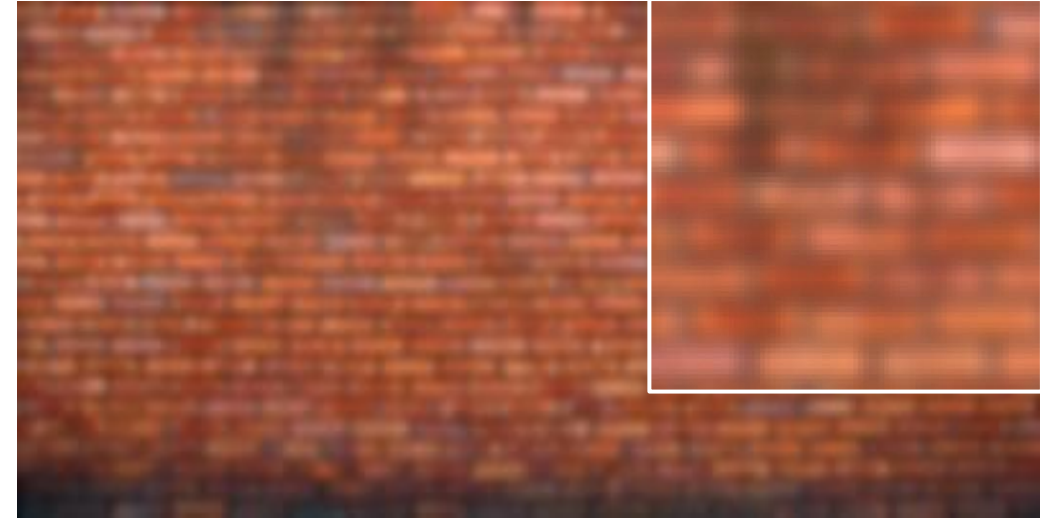


Gaussian vs box filtering

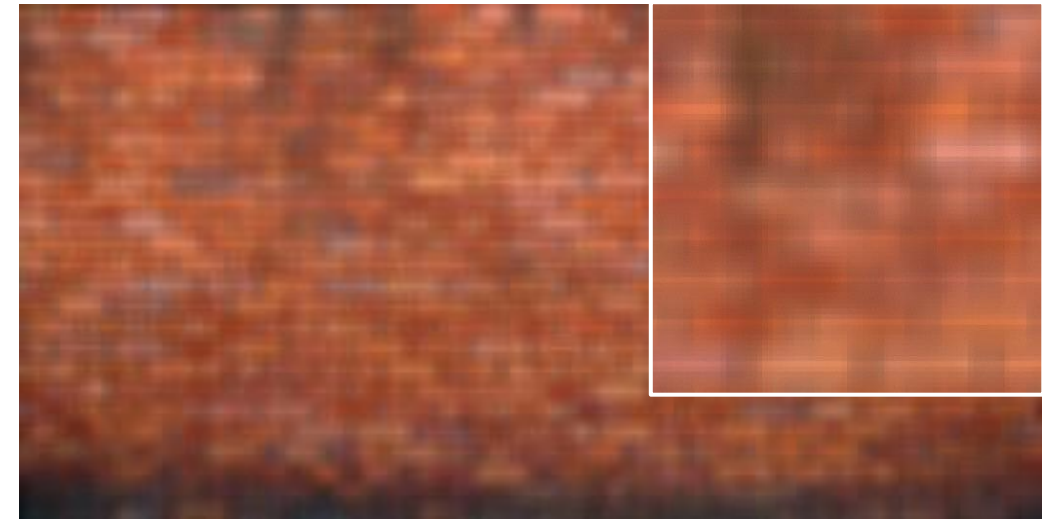


original

Which blur do you like better?



7x7 Gaussian

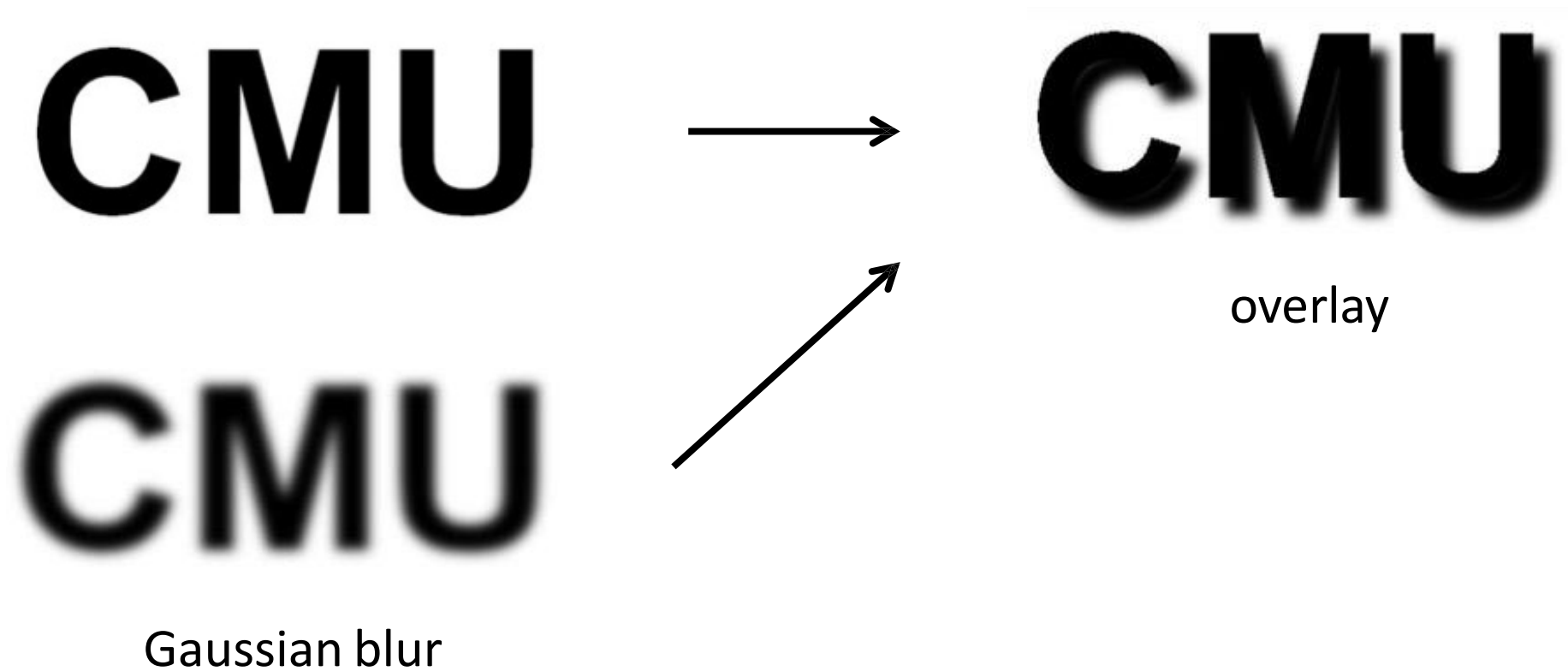


7x7 box

How would you create a soft shadow effect?

CMU → **CMU**

How would you create a soft shadow effect?



Other filters

input



filter

0	0	0
0	1	0
0	0	0

output

?

Other filters

input



filter

0	0	0
0	1	0
0	0	0

output



unchanged

Other filters

input



filter

0	0	0
0	1	0
0	0	0

output



unchanged

input



filter

0	0	0
0	0	1
0	0	0

output

?

Other filters

input



filter

0	0	0
0	1	0
0	0	0

output



unchanged

input



filter

0	0	0
0	0	1
0	0	0

output



shift to left
by one

Other filters

input



filter

0	0	0
0	2	0
0	0	0

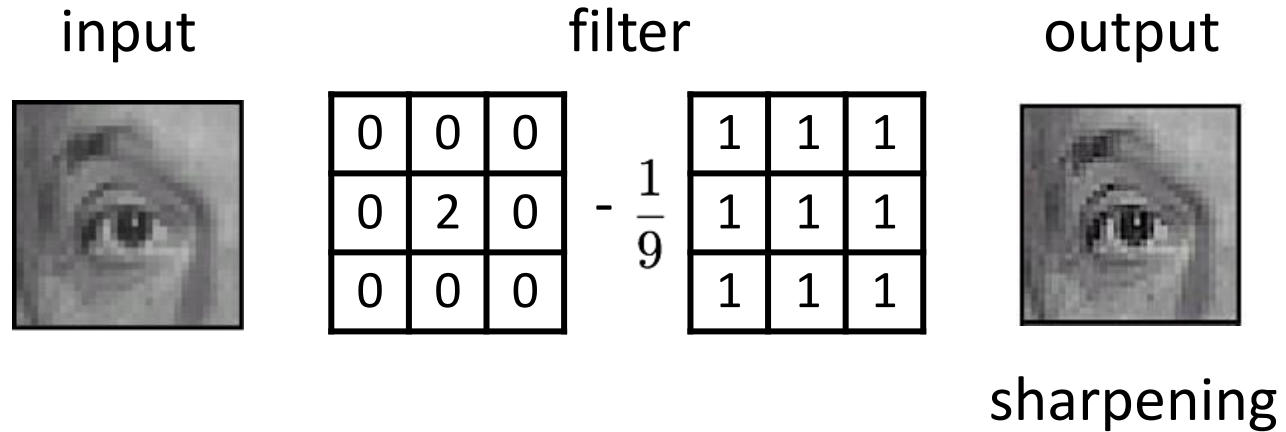
- $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

output

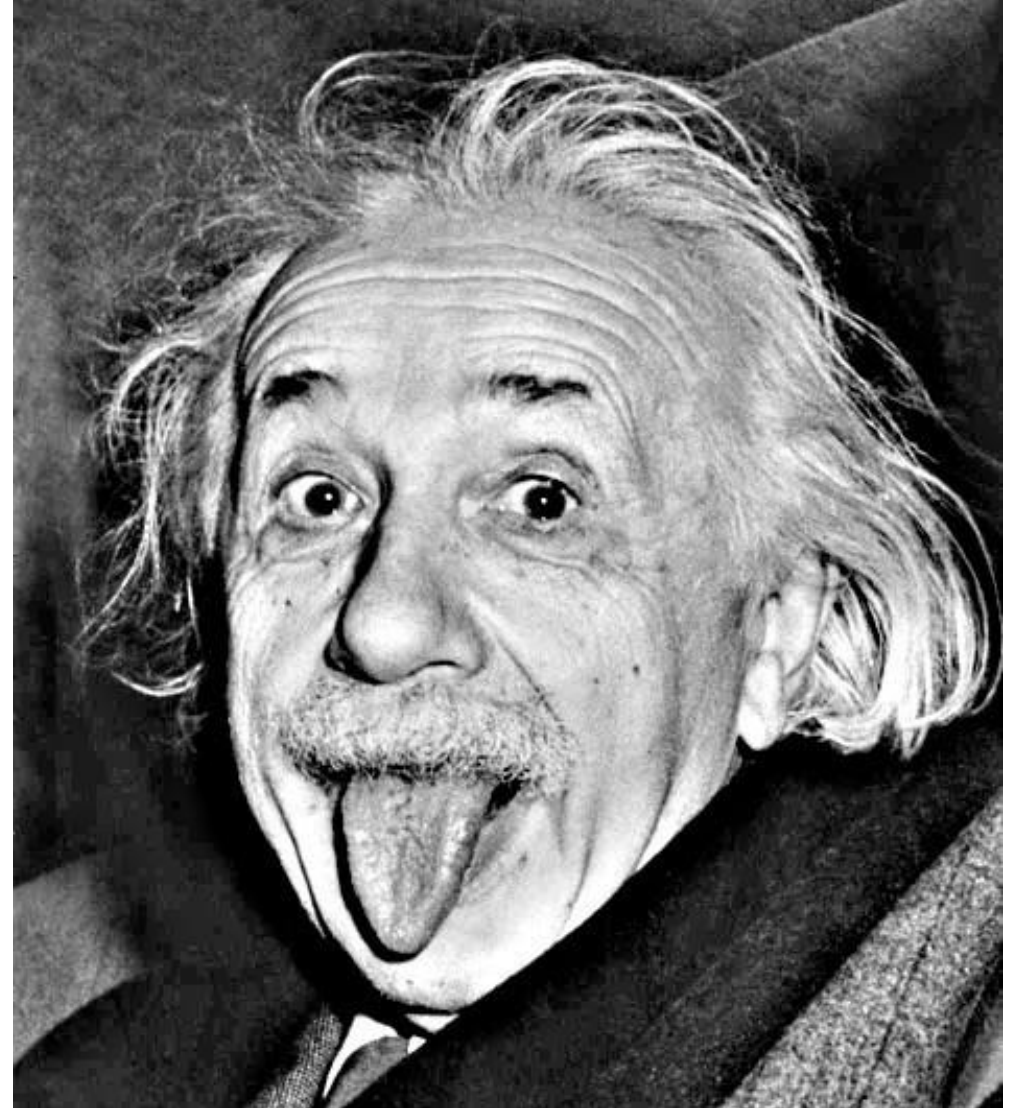
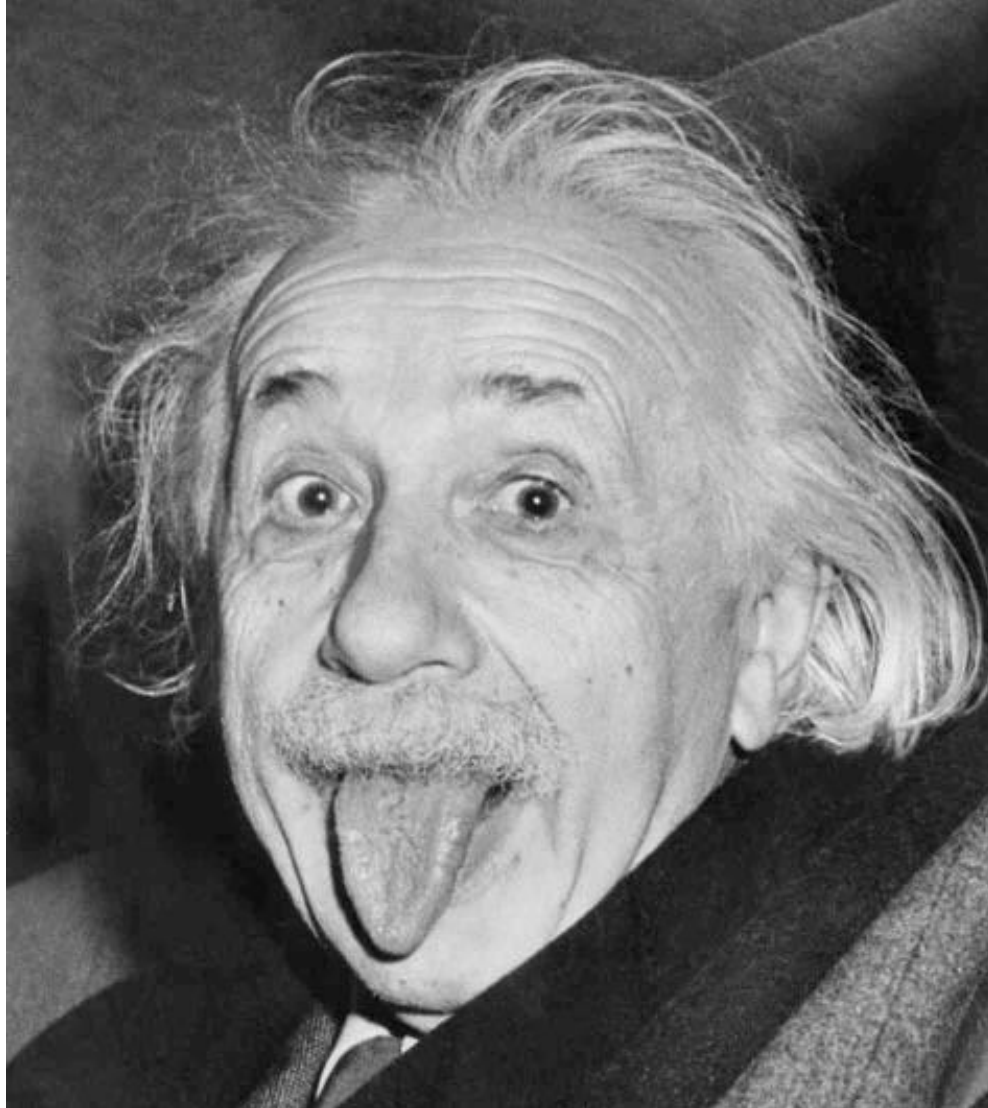
?

Other filters



- do nothing for flat areas
- stress intensity peaks

Sharpening examples



Sharpening examples



Sharpening examples



Sharpening examples



Do not overdo it with sharpening



original



sharpened



oversharpened

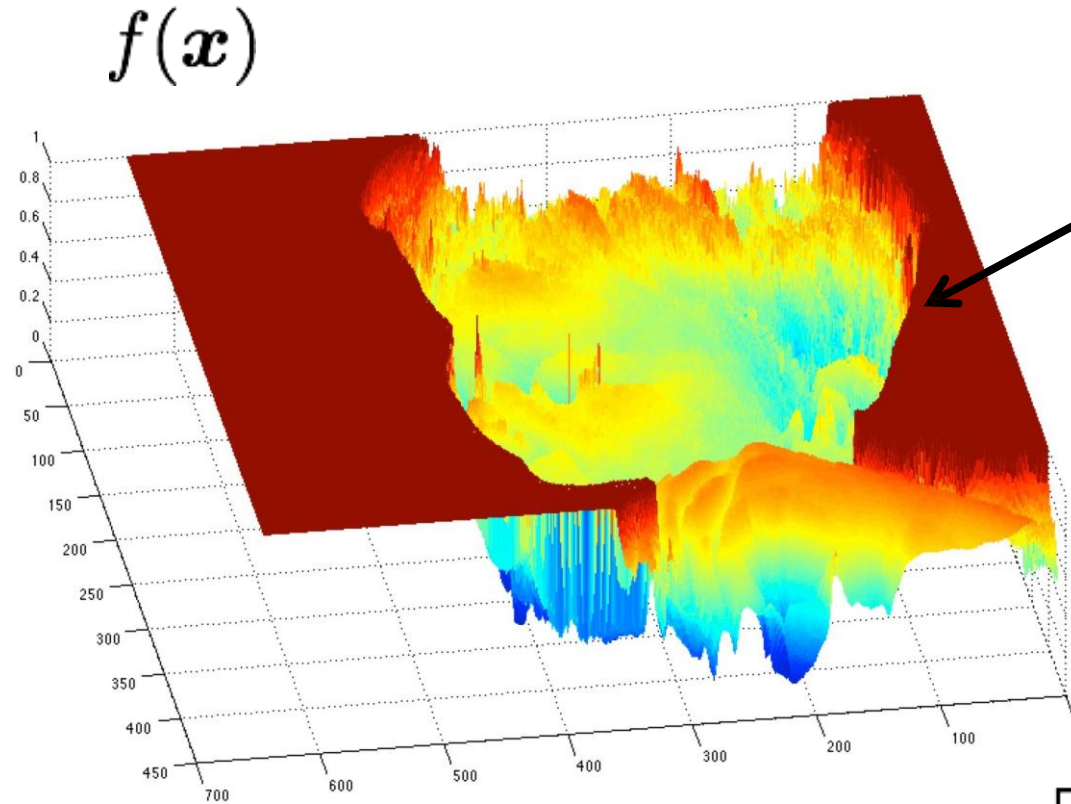
What is wrong in this image?

Image gradients

What are image edges?



grayscale image



Very sharp
discontinuities
in intensity.

domain $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

Detecting edges

How would you go about detecting edges in an image (i.e., discontinuities in a function)?

Detecting edges

How would you go about detecting edges in an image (i.e., discontinuities in a function)?

✓ You take derivatives: derivatives are large at discontinuities.

How do you differentiate a discrete image (or any other discrete signal)?

Detecting edges

How would you go about detecting edges in an image (i.e., discontinuities in a function)?

✓ You take derivatives: derivatives are large at discontinuities.

How do you differentiate a discrete image (or any other discrete signal)?

✓ You use finite differences.

Finite differences

High-school reminder: definition of a derivative using forward difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Finite differences

High-school reminder: definition of a derivative using forward difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Alternative: use central difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+0.5h) - f(x-0.5h)}{h}$$

For discrete signals: Remove limit and set $h = 2$

$$f'(x) = \frac{f(x+1) - f(x-1)}{2}$$

What convolution kernel does this correspond to?

Finite differences

High-school reminder: definition of a derivative using forward difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Alternative: use central difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+0.5h) - f(x-0.5h)}{h}$$

For discrete signals: Remove limit and set $h = 2$

$$f'(x) = \frac{f(x+1) - f(x-1)}{2}$$

-1	0	1	?
1	0	-1	?

Finite differences

High-school reminder: definition of a derivative using forward difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Alternative: use central difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+0.5h) - f(x-0.5h)}{h}$$

For discrete signals: Remove limit and set $h = 2$

$$f'(x) = \frac{f(x+1) - f(x-1)}{2}$$

1D derivative filter

1	0	-1
---	---	----

The Sobel filter

1	0	-1
2	0	-2
1	0	-1

Sobel filter

=

1
2
1

What filter
is this?

*

1	0	-1
---	---	----

1D derivative
filter

The Sobel filter

1	0	-1
2	0	-2
1	0	-1

Sobel filter

=

1
2
1

Blurring

*

1	0	-1
---	---	----

1D derivative
filter

Does this filter return large responses on vertical or horizontal lines?

The Sobel filter

Horizontal Sober filter:

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

What does the vertical Sobel filter look like?

The Sobel filter

Horizontal Sobel filter:

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

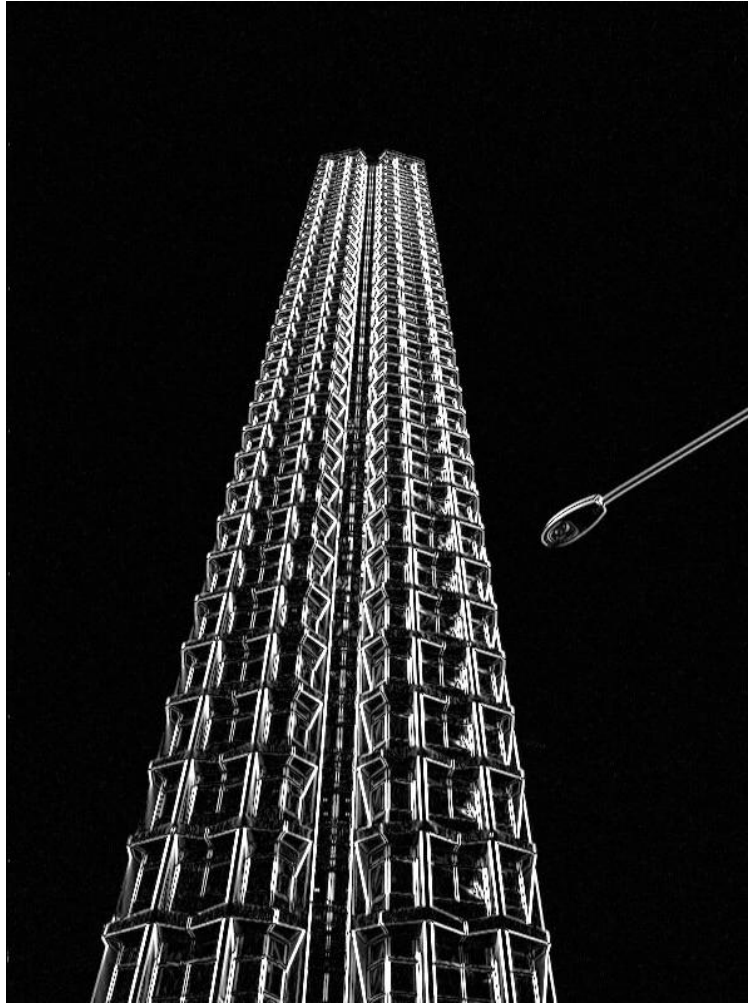
Vertical Sobel filter:

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Sobel filter example



original



which Sobel filter?

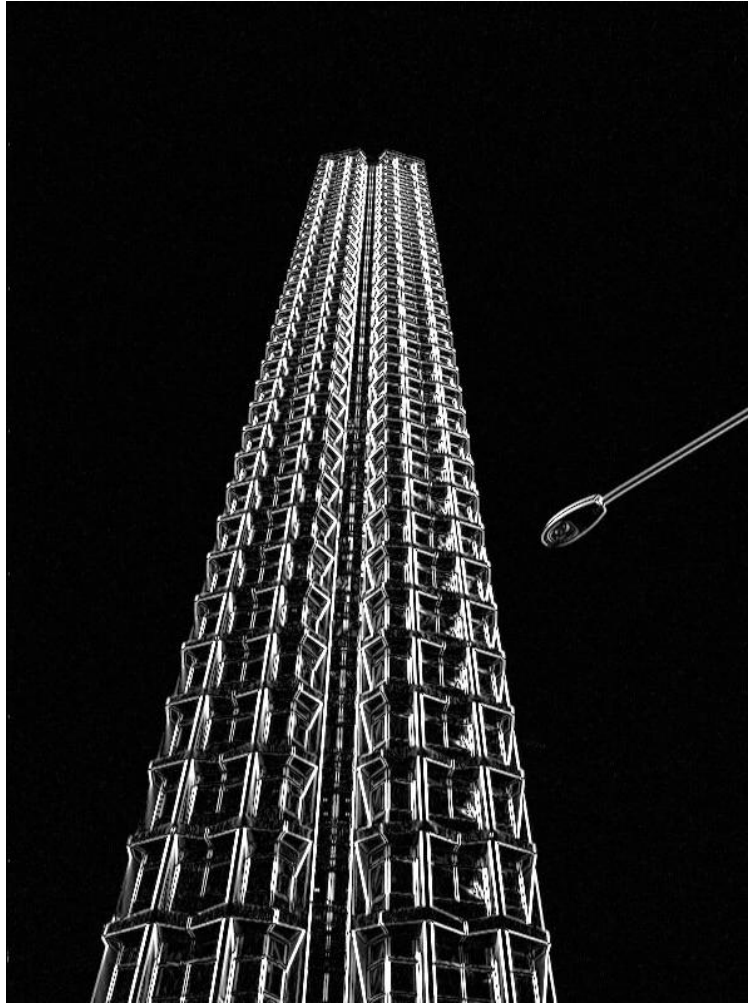


which Sobel filter?

Sobel filter example



original

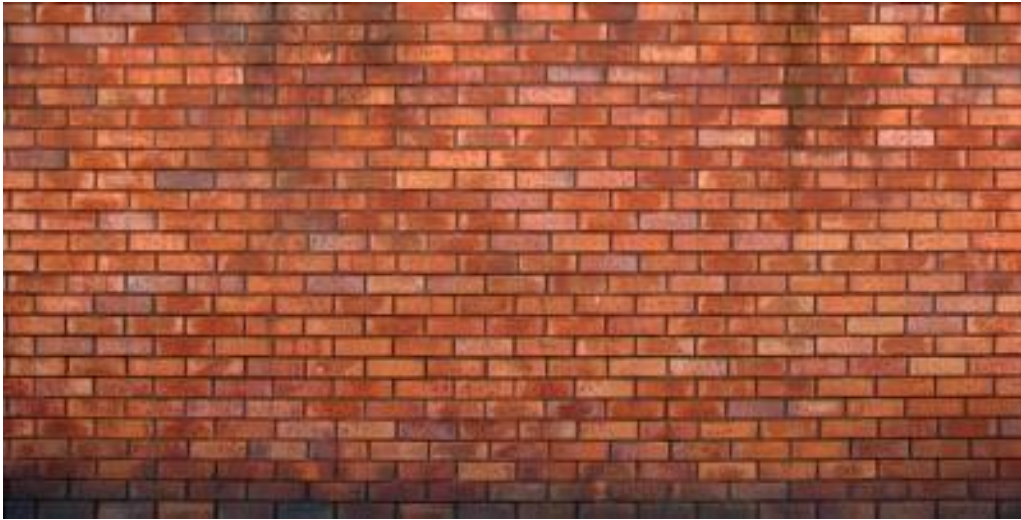


horizontal Sobel filter



vertical Sobel filter

Sobel filter example



original



horizontal Sobel filter



vertical Sobel filter

Several derivative filters

Sobel

1	0	-1
2	0	-2
1	0	-1

1	2	1
0	0	0
-1	-2	-1

Scharr

3	0	-3
10	0	-10
3	0	-3

3	10	3
0	0	0
-3	-10	-3

Prewitt

1	0	-1
1	0	-1
1	0	-1

1	1	1
0	0	0
-1	-1	-1

Roberts

0	1
-1	0

1	0
0	-1

- How are the other filters derived and how do they relate to the Sobel filter?
- How would you derive a derivative filter that is larger than 3x3?

Computing image gradients

1. Select your favorite derivative filters.

$$S_x = \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 2 & 0 & -2 \\ \hline 1 & 0 & -1 \\ \hline \end{array}$$

$$S_y = \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline -1 & -2 & -1 \\ \hline \end{array}$$

Computing image gradients

1. Select your favorite derivative filters.

$$S_x = \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 2 & 0 & -2 \\ \hline 1 & 0 & -1 \\ \hline \end{array}$$

$$S_y = \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline -1 & -2 & -1 \\ \hline \end{array}$$

2. Convolve with the image to compute derivatives.

$$\frac{\partial f}{\partial x} = S_x \otimes f$$

$$\frac{\partial f}{\partial y} = S_y \otimes f$$

Computing image gradients

1. Select your favorite derivative filters.

$$\mathbf{S}_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\mathbf{S}_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

2. Convolve with the image to compute derivatives.

$$\frac{\partial f}{\partial x} = \mathbf{S}_x \otimes f$$

$$\frac{\partial f}{\partial y} = \mathbf{S}_y \otimes f$$

3. Form the image gradient, and compute its direction and amplitude.

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

gradient

$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

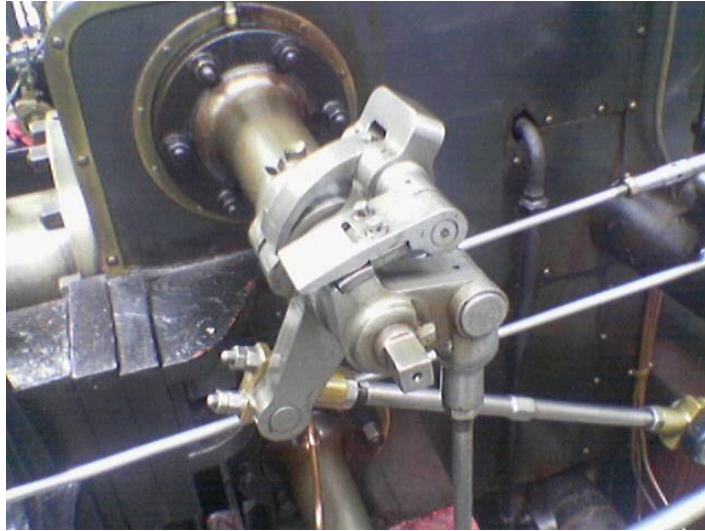
direction

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$

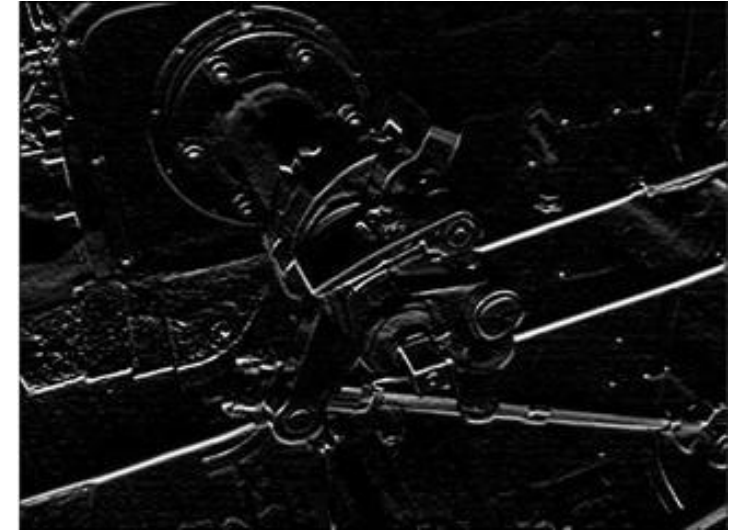
amplitude

Image gradient example

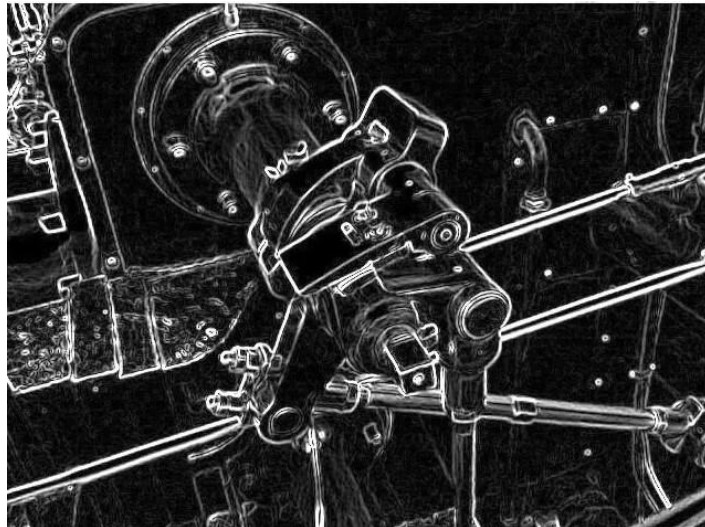
original



vertical
derivative



gradient
amplitude



horizontal
derivative



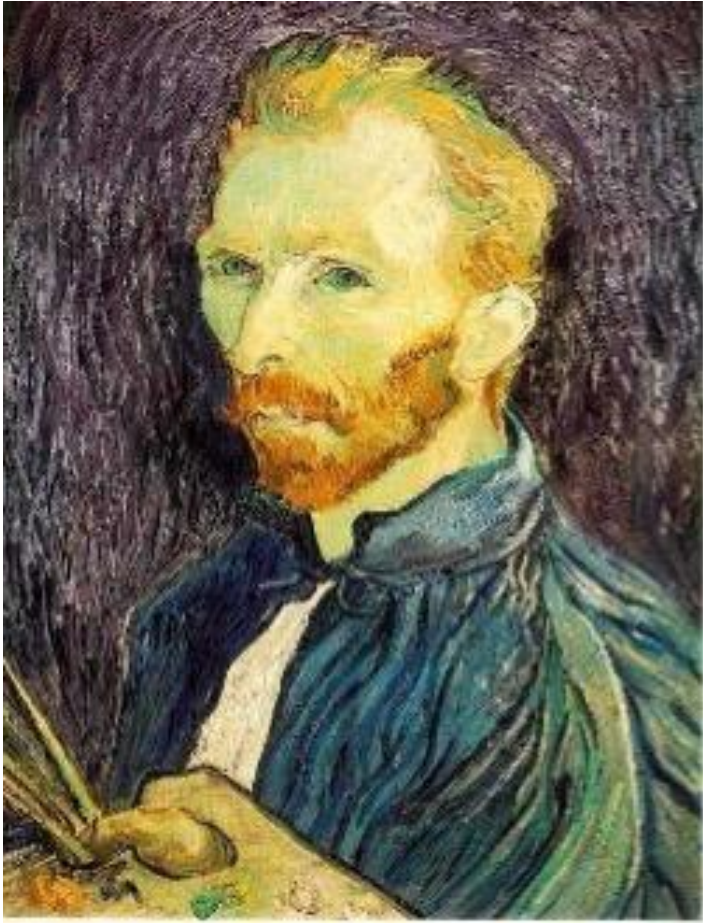
How does the gradient direction relate to these edges?

Image downsampling

A close-up of Vincent van Gogh's 'Self-Portrait with Bandaged Ear'. The painting shows a man with a pale, yellowish face, looking slightly to the left. He has a thick, reddish-brown beard and mustache. His hair is a mix of yellow and green. He is wearing a dark blue jacket. The background is a dark, textured brown. The brushstrokes are visible and expressive.

**This image is too big to fit on the screen.
How would you reduce it to half its size?**

Naïve image downsampling



1/2

Throw away half the rows and columns

delete even rows
delete even columns



1/4

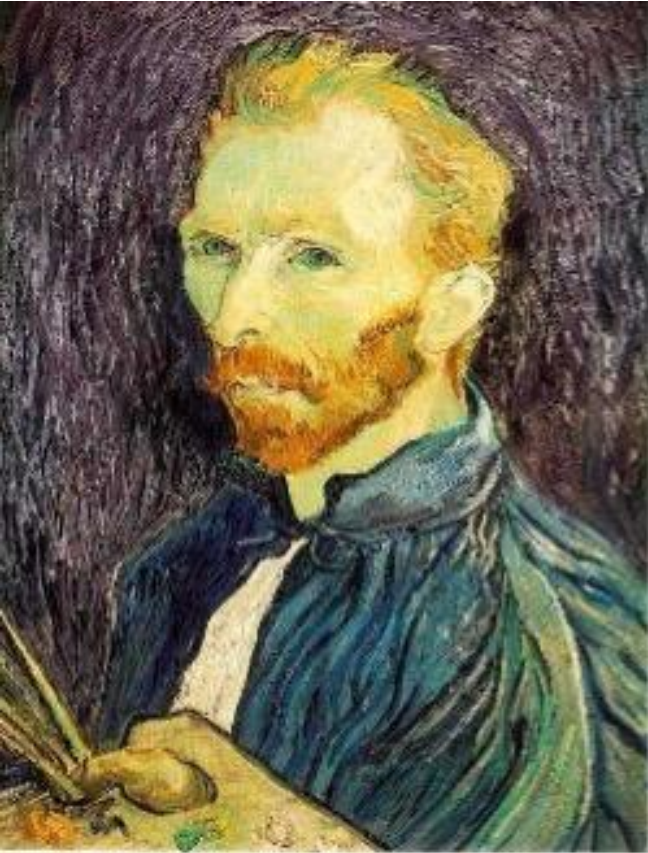
delete even rows
delete even columns



1/8

What is the problem with this approach?

Naïve image downsampling



$1/2$



$1/4$ (2x zoom)

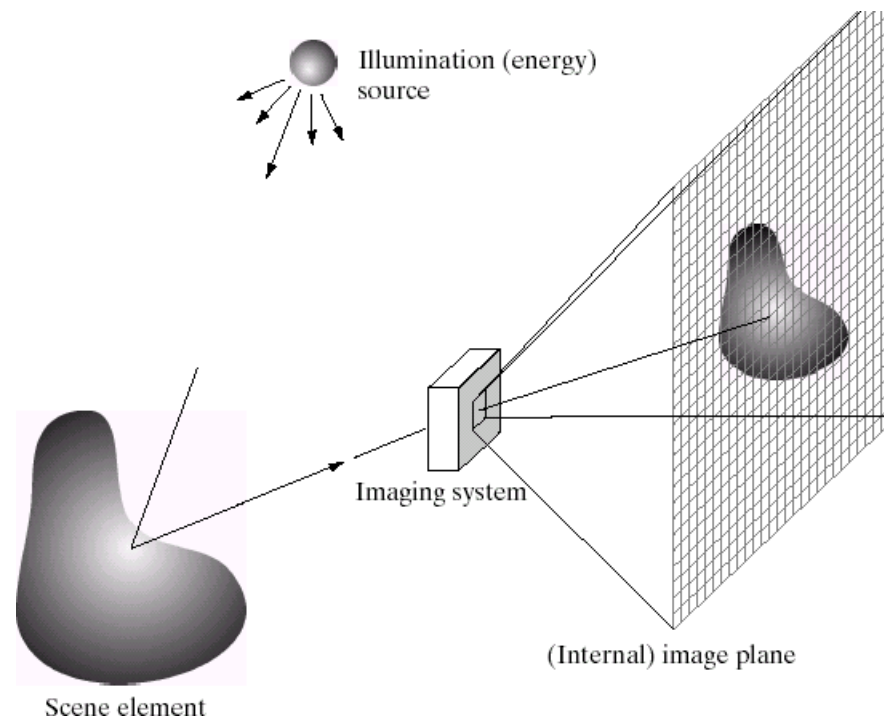


$1/8$ (4x zoom)

Why is the $1/8$ image so pixelated (and do you know what this effect is called)?

Aliasing

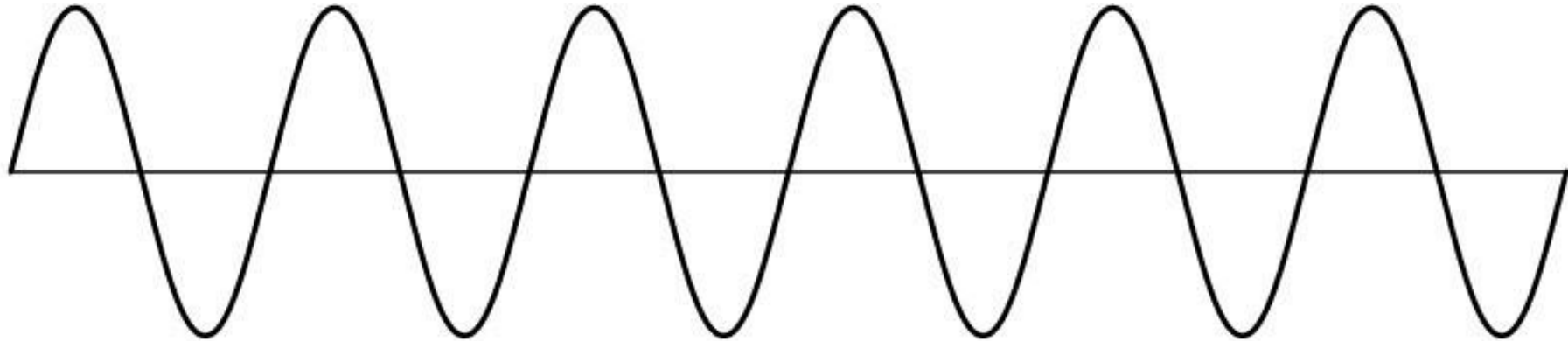
Reminder



Images are a *discrete*, or *sampled*, representation of a *continuous* world

Sampling

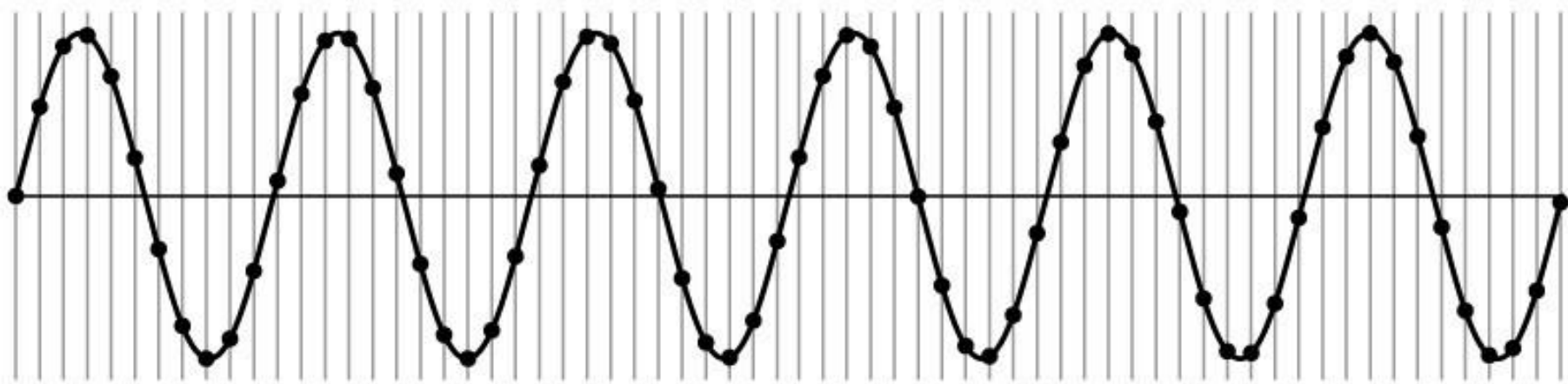
Very simple example: a sine wave



How would you discretize this signal?

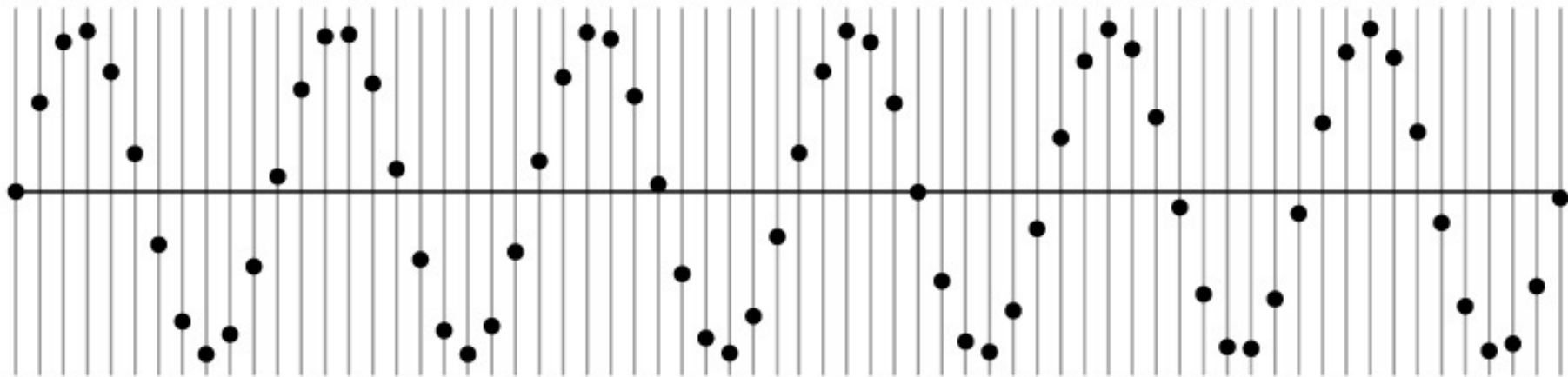
Sampling

Very simple example: a sine wave



Sampling

Very simple example: a sine wave

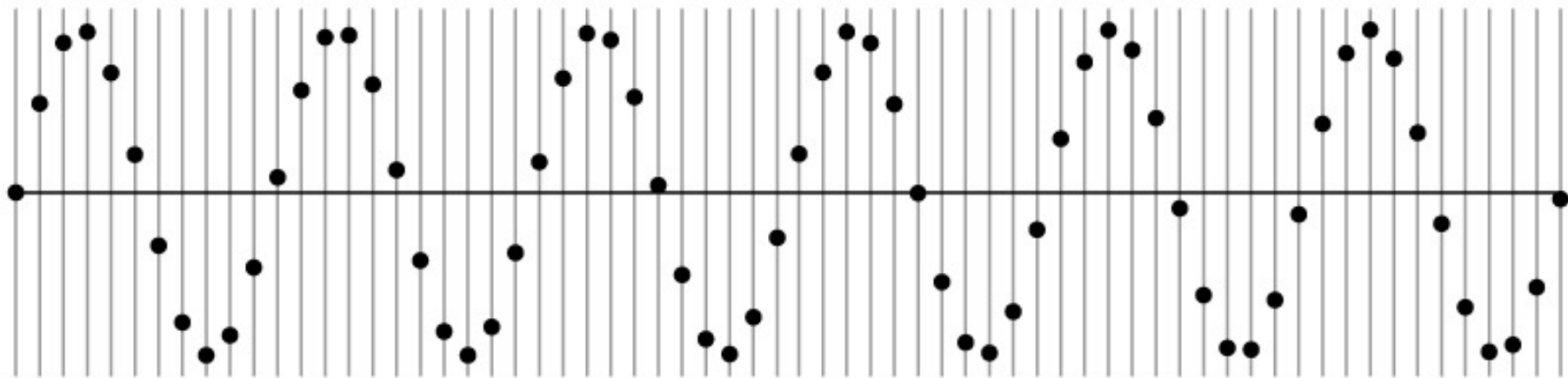


How many samples should I take?

Can I take as *many* samples as I want?

Sampling

Very simple example: a sine wave

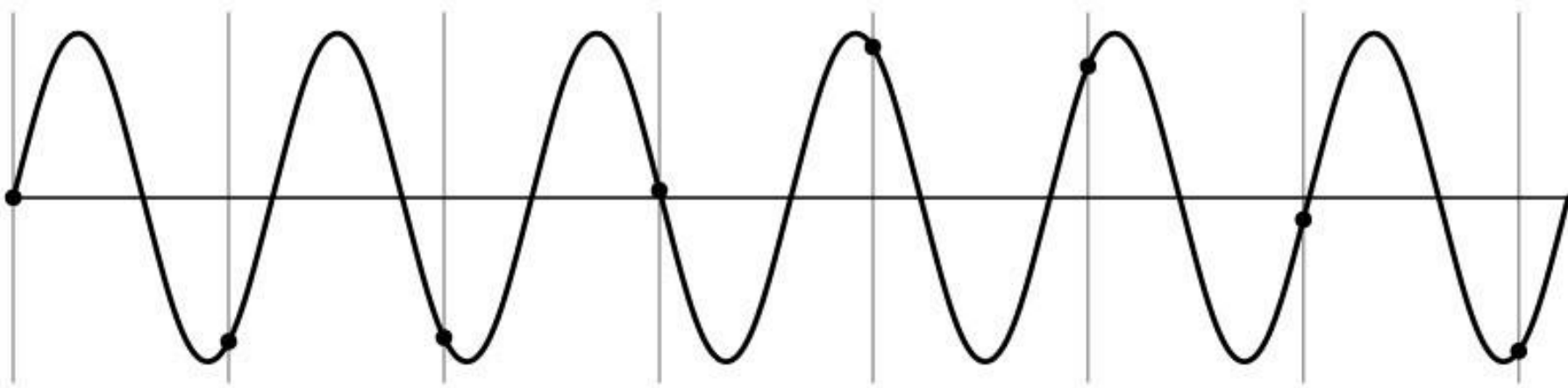


How many samples should I take?

Can I take as *few* samples as I want?

Undersampling

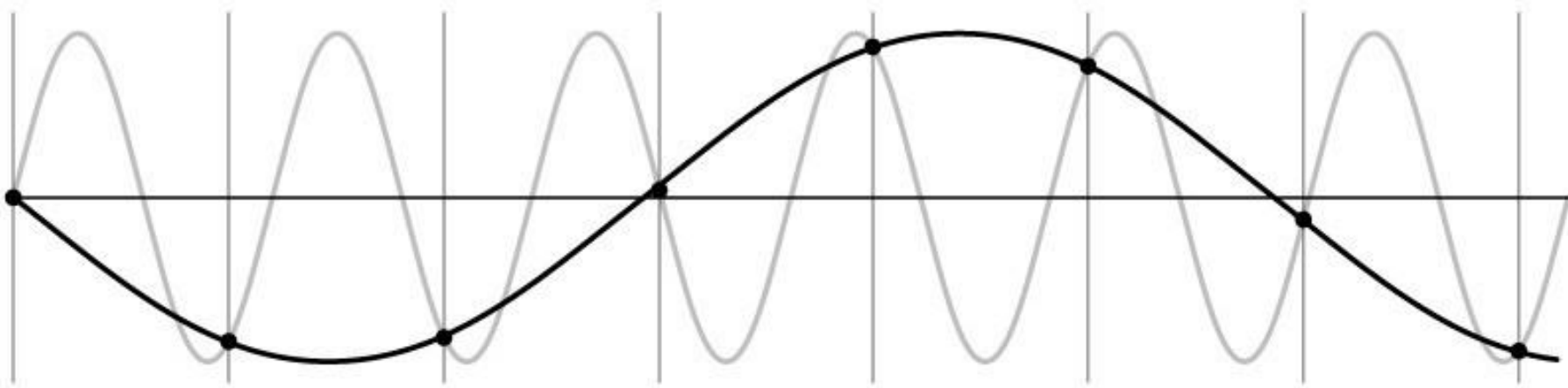
Very simple example: a sine wave



Unsurprising effect: information is lost.

Undersampling

Very simple example: a sine wave

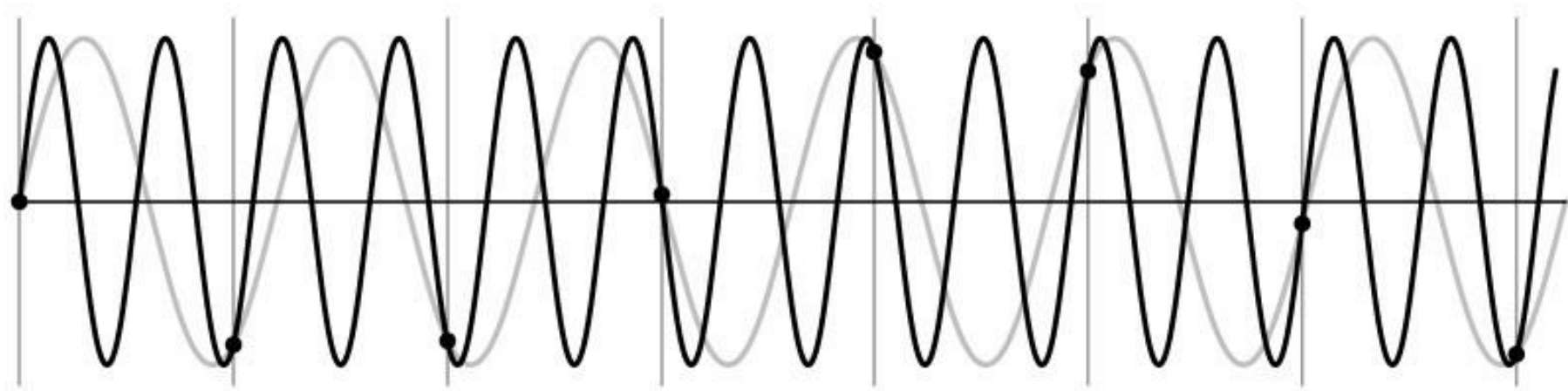


Unsurprising effect: information is lost.

Surprising effect: can confuse the signal with one of *lower* frequency.

Undersampling

Very simple example: a sine wave



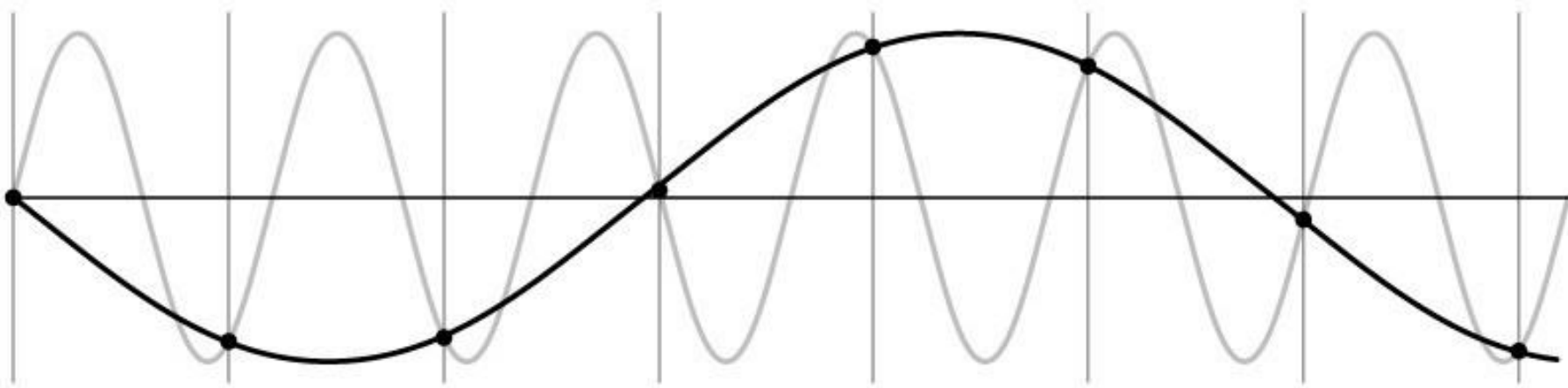
Unsurprising effect: information is lost.

Surprising effect: can confuse the signal with one of *lower* frequency.

Note: we could always confuse the signal with one of *higher* frequency.

Aliasing

Fancy term for: *Undersampling can disguise a signal as one of a lower frequency*

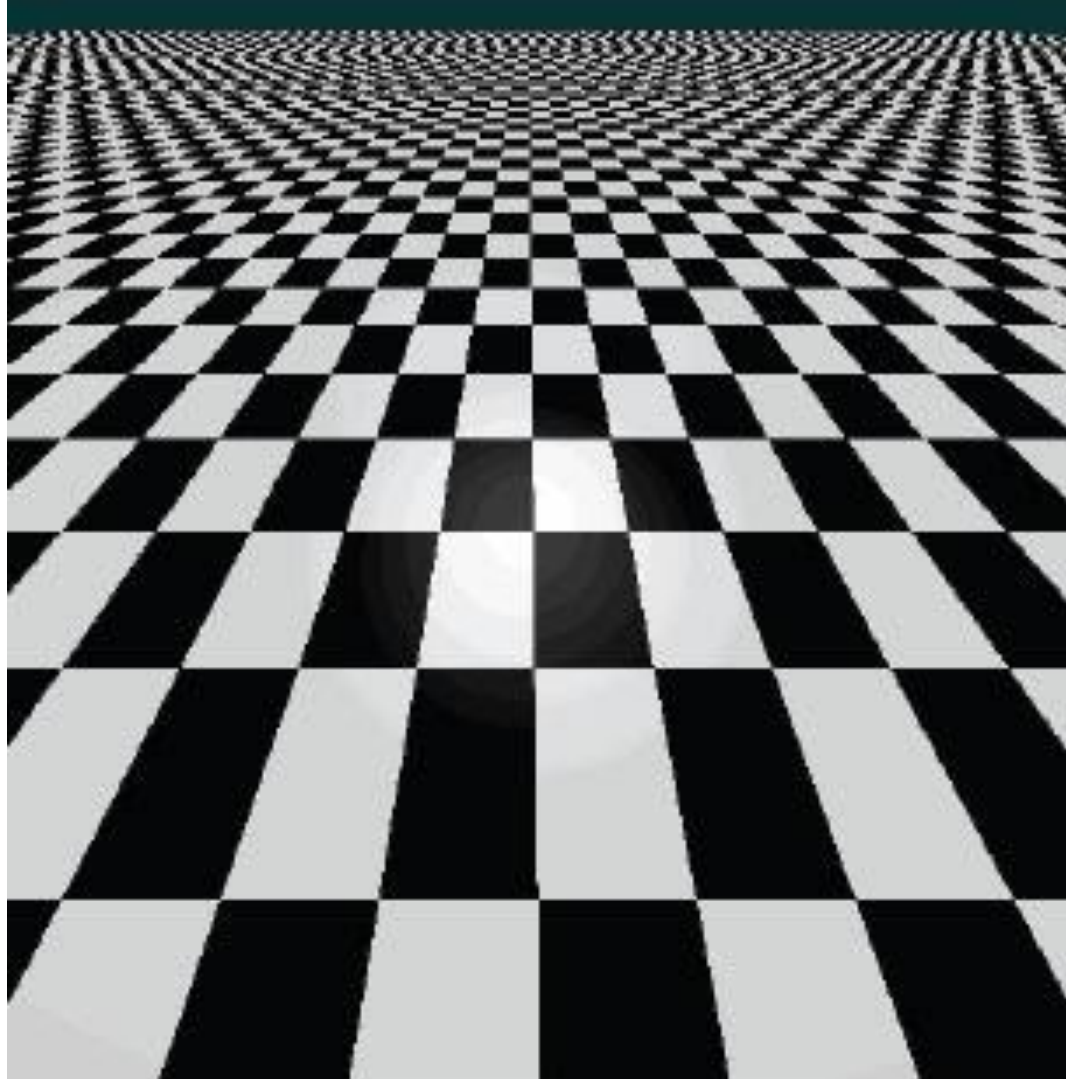


Unsurprising effect: information is lost.

Surprising effect: can confuse the signal with one of *lower* frequency.

Note: we could always confuse the signal with one of *higher* frequency.

Aliasing in textures



Aliasing in photographs

This is also known as “moire”

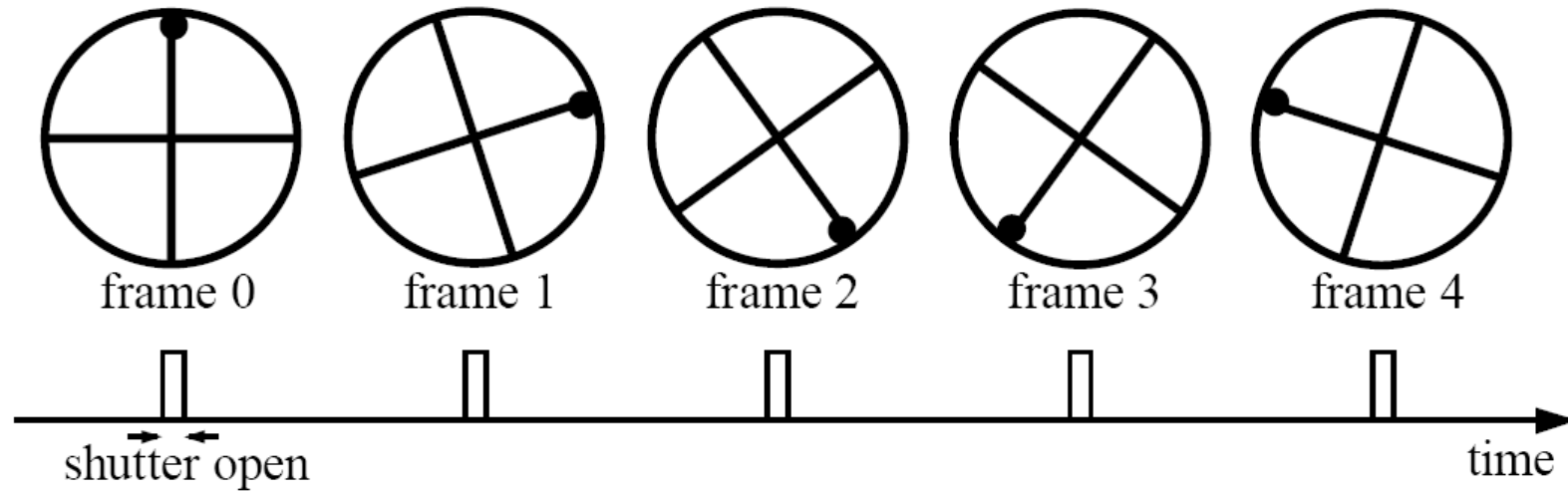


Temporal aliasing

Imagine a spoked wheel moving to the right (rotating clockwise).

Mark wheel with dot so we can see what's happening.

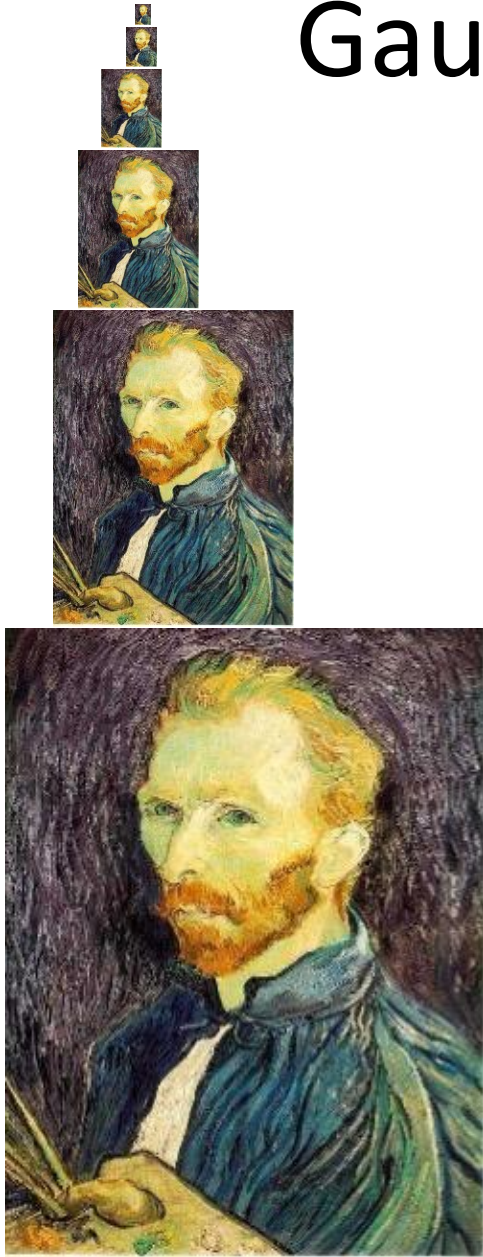
If camera shutter is only open for a fraction of a frame time (frame time = $1/30$ sec. for video, $1/24$ sec. for film):



Without dot, wheel appears to be rotating slowly backwards!
(counterclockwise)

Gaussian image pyramid

Gaussian image pyramid

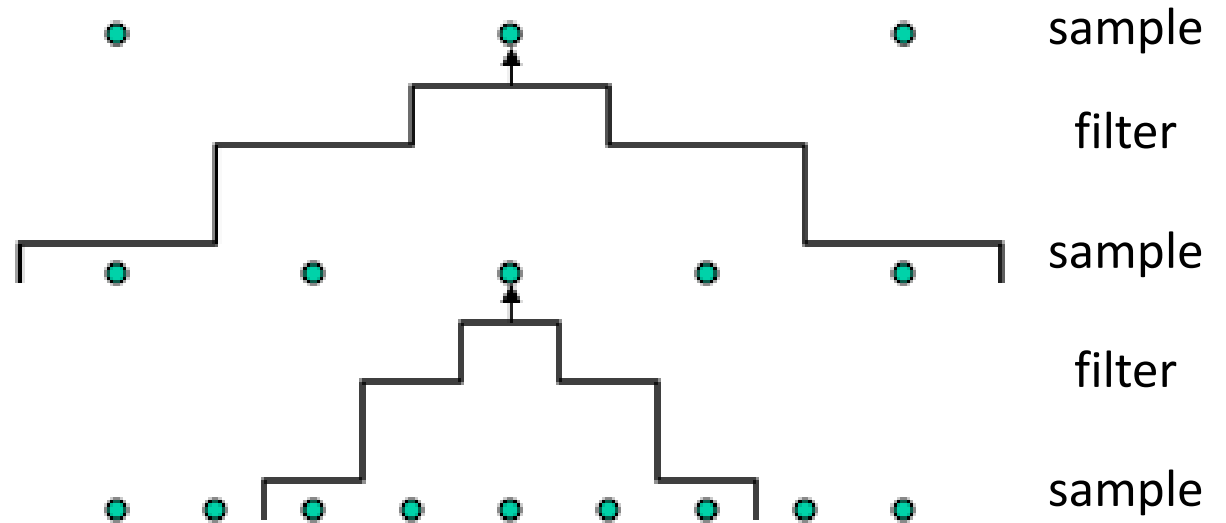


The name of this sequence of subsampled images

Constructing a Gaussian pyramid

Algorithm

```
repeat:  
    filter  
    subsample  
until min resolution reached
```

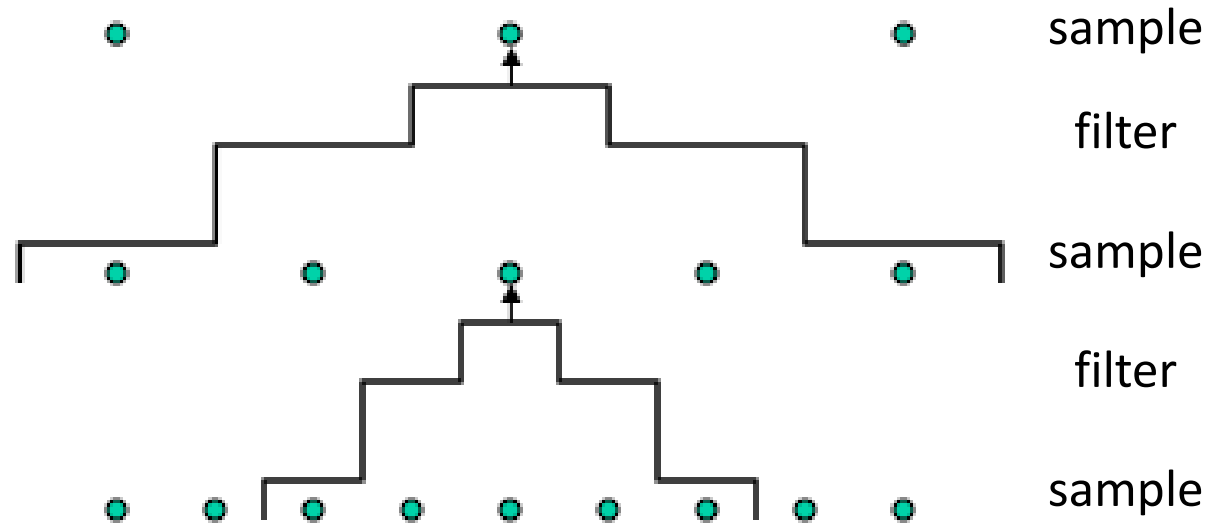


Question: How much bigger than the original image is the whole pyramid?

Constructing a Gaussian pyramid

Algorithm

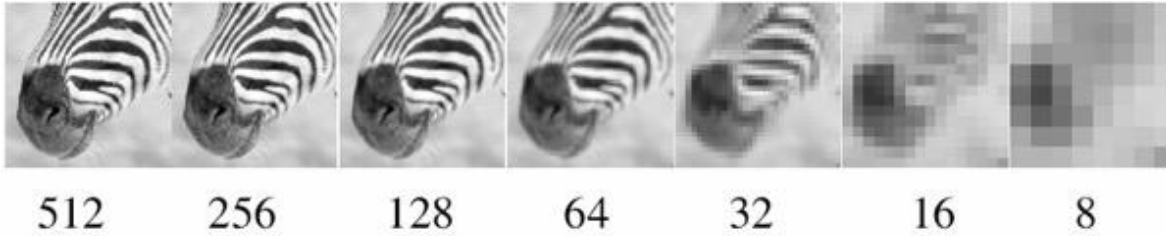
```
repeat:  
    filter  
    subsample  
until min resolution reached
```



Question: How much bigger than the original image is the whole pyramid?

Answer: Just $\frac{4}{3}$ times the size of the original image! (How did I come up with this number?)

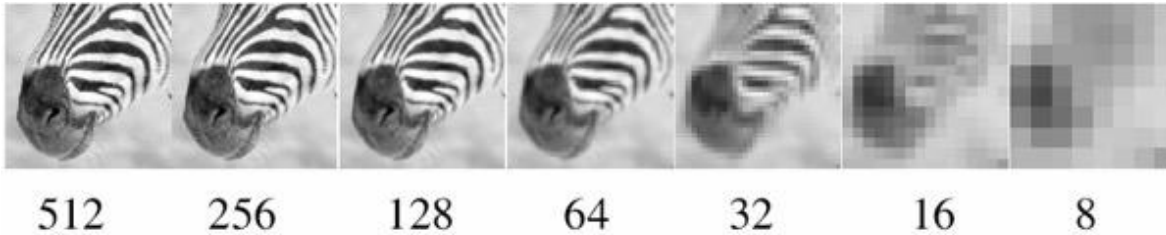
Some properties of the Gaussian pyramid



What happens to the details of the image?



Some properties of the Gaussian pyramid

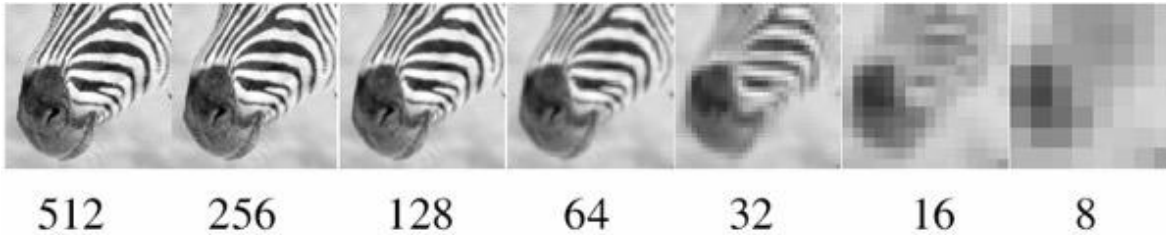


What happens to the details of the image?

- They get smoothed out as we move to higher levels.

What is preserved at the higher levels?

Some properties of the Gaussian pyramid



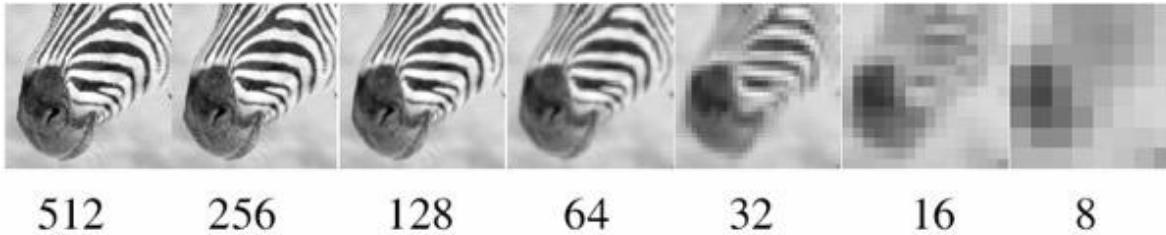
What happens to the details of the image?

- They get smoothed out as we move to higher levels.

What is preserved at the higher levels?

- Mostly large uniform regions in the original

Some properties of the Gaussian pyramid



What happens to the details of the image?

- They get smoothed out as we move to higher levels.

What is preserved at the higher levels?

- Mostly large uniform regions in the original

Blurring is lossy



level 0



level 1 (before downsampling)

What does the residual look like?

Blurring is lossy



level 0

-



level 1 (before downsampling)

=

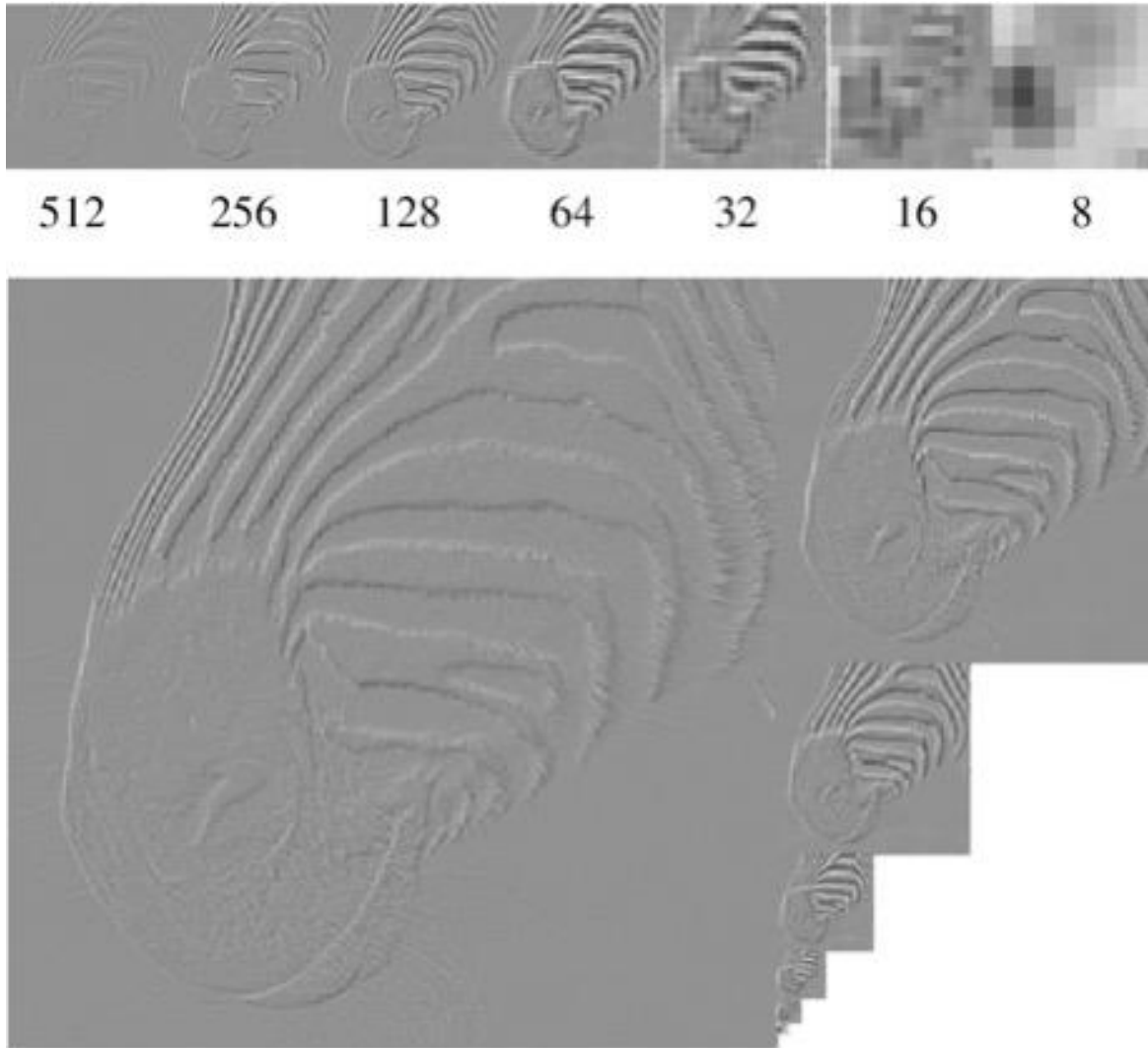


residual

Can we make a pyramid that is lossless?

Laplacian image pyramid

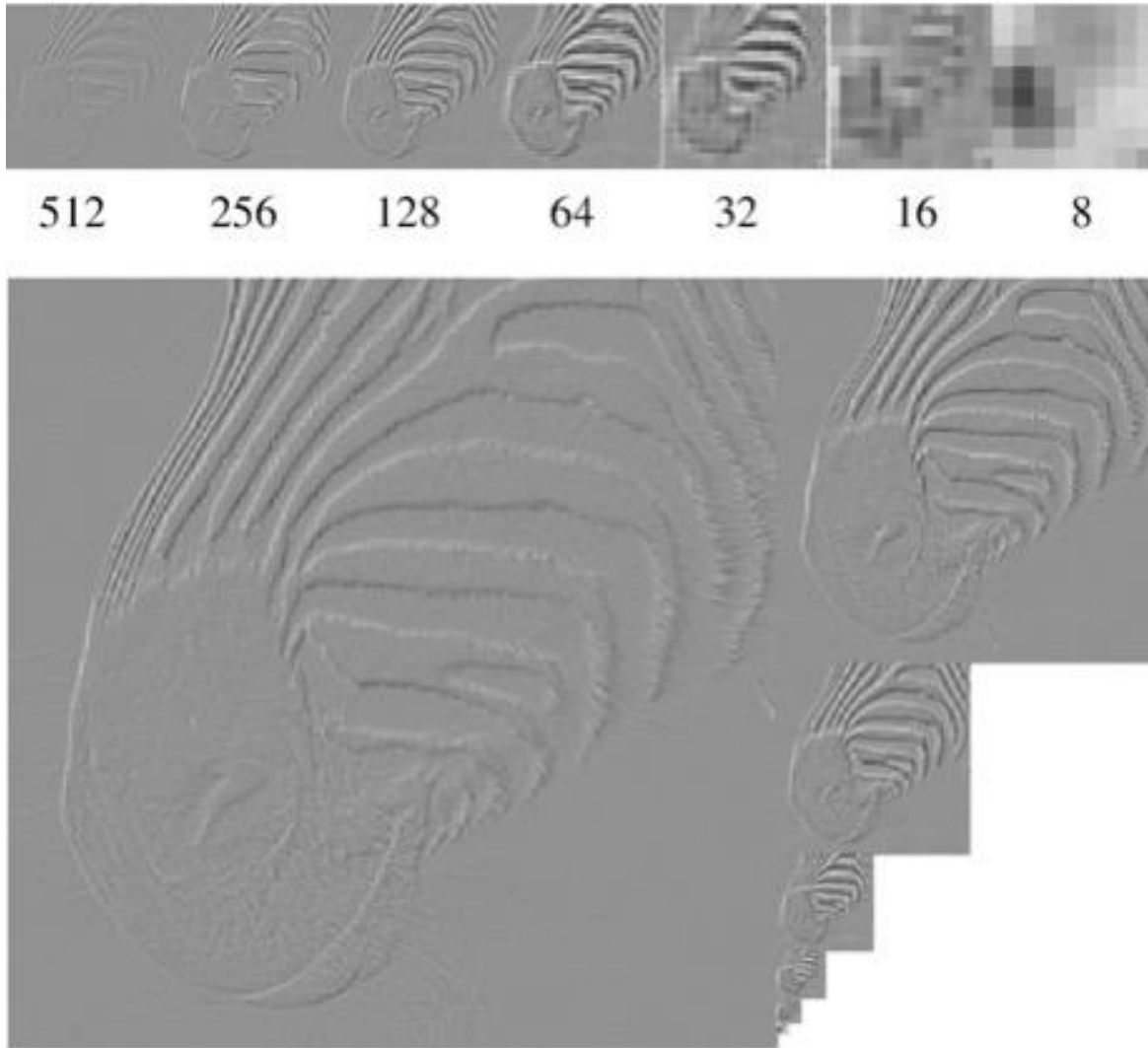
Laplacian image pyramid



At each level, retain the residuals instead of the blurred images themselves.

Can we reconstruct the original image using the pyramid?

Laplacian image pyramid



At each level, retain the residuals instead of the blurred images themselves.

Can we reconstruct the original image using the pyramid?

- Yes we can!

What do we need to store to be able to reconstruct the original image?

Let's start by looking at just one level



level 0

=



level 1 (upsampled)

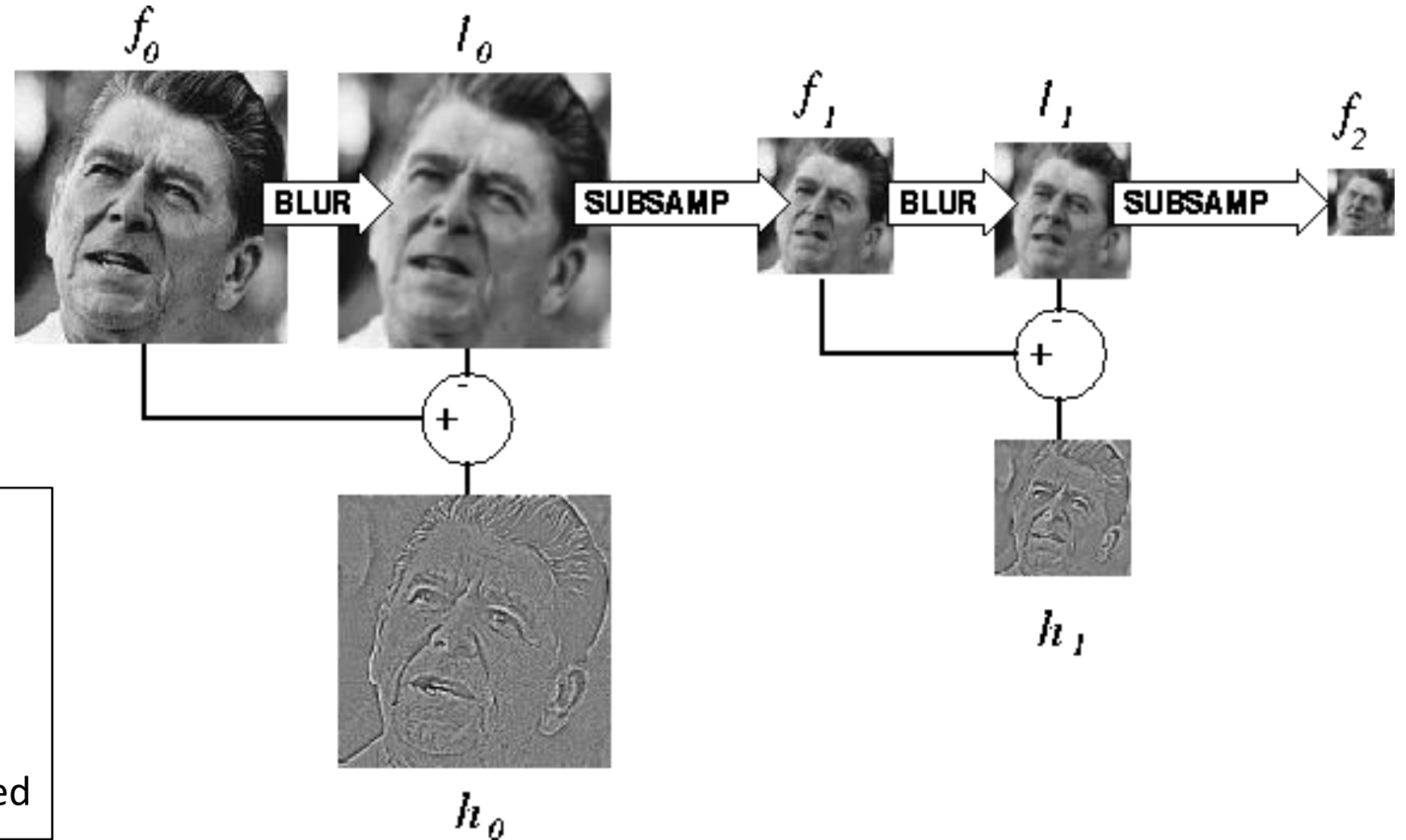
+



residual

Does this mean we need to store both residuals and the blurred copies of the original?

Constructing a Laplacian pyramid



Algorithm

repeat:

filter

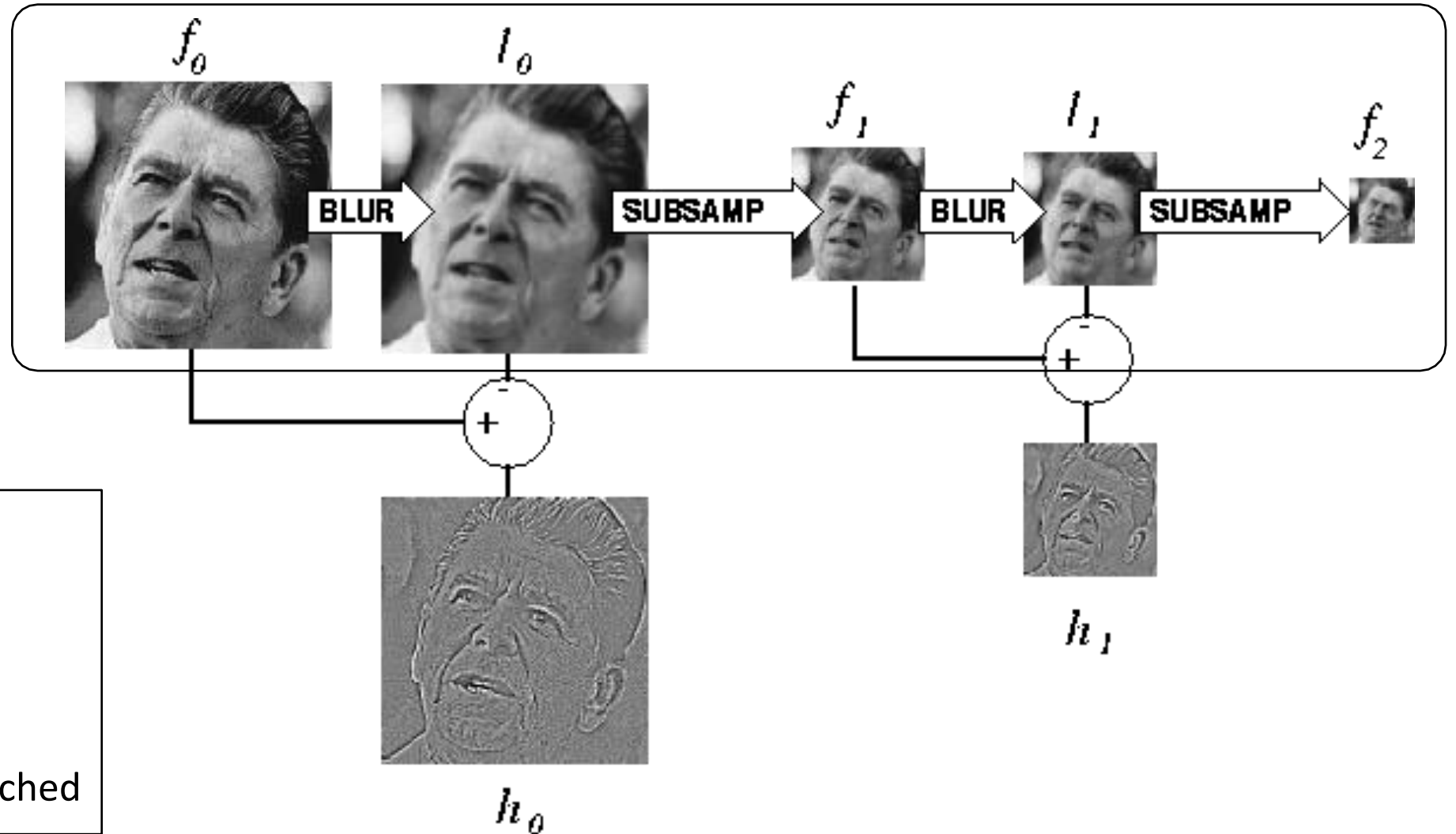
compute residual

subsample

until min resolution reached

Constructing a Laplacian pyramid

What is this part?



Algorithm

repeat:

filter

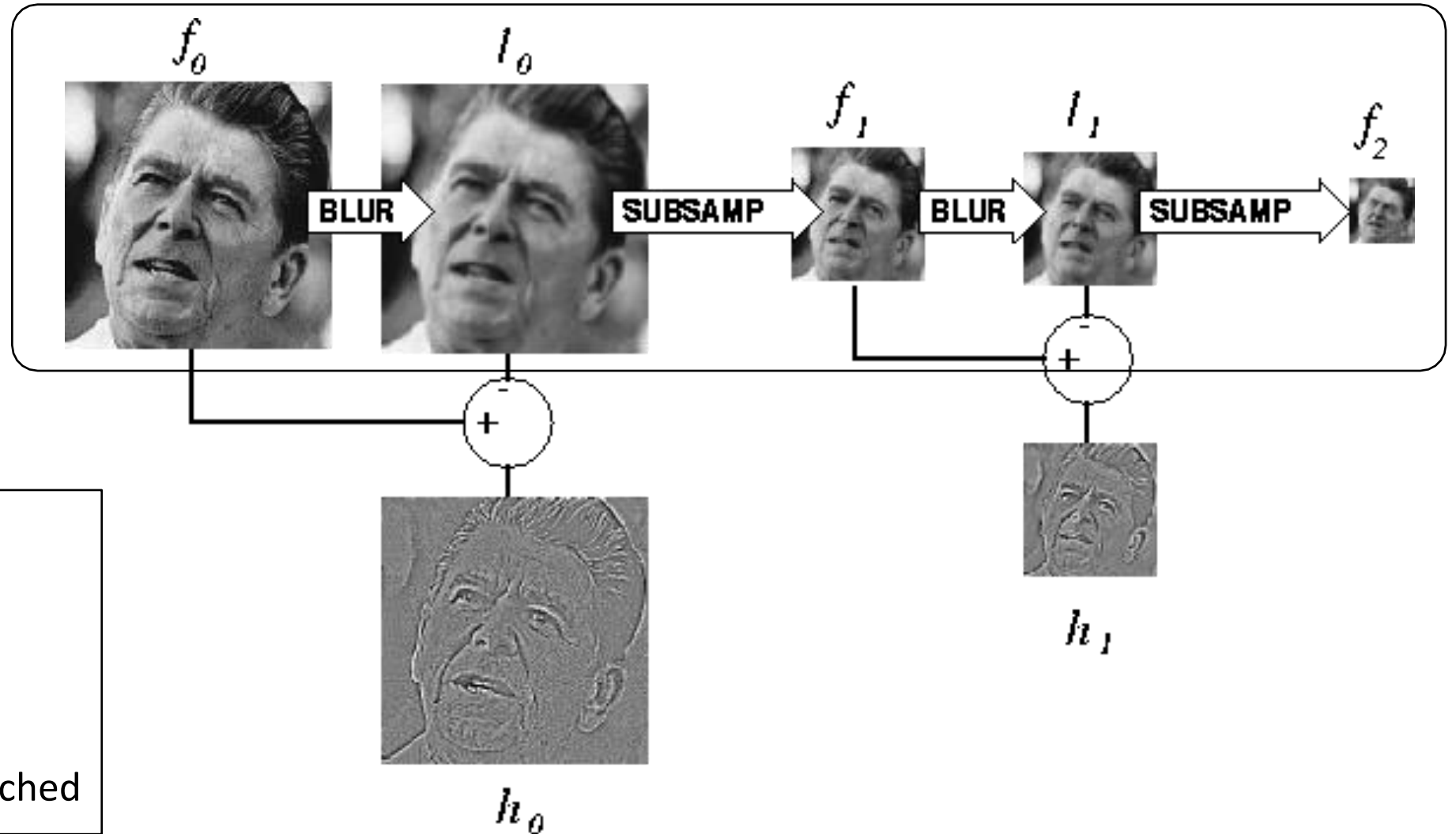
compute residual

subsample

until min resolution reached

Constructing a Laplacian pyramid

It's a Gaussian pyramid.



Algorithm

repeat:

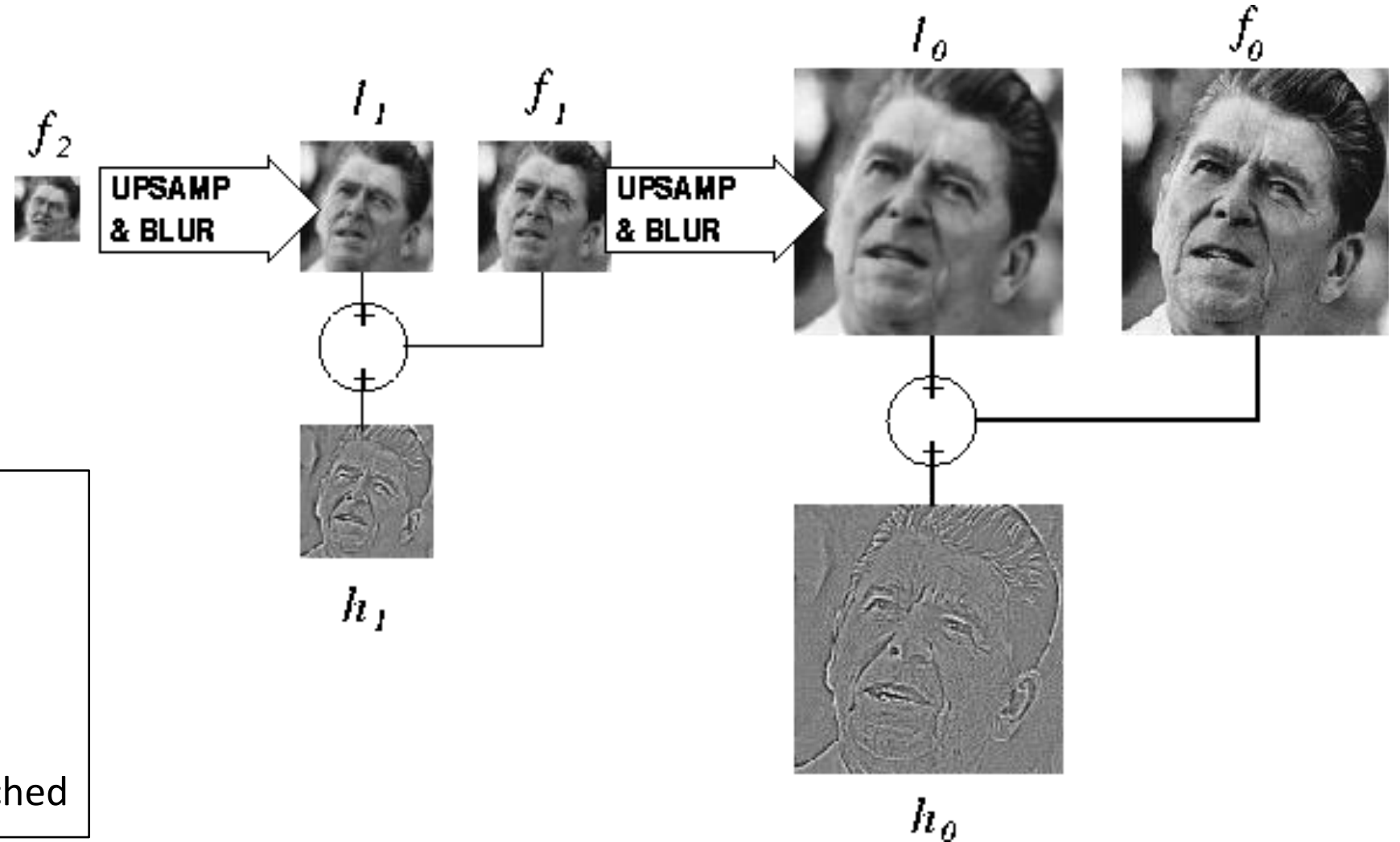
filter

compute residual

subsample

until min resolution reached

Reconstructing the original image



Algorithm

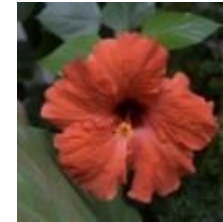
repeat:

upsample

sum with residual

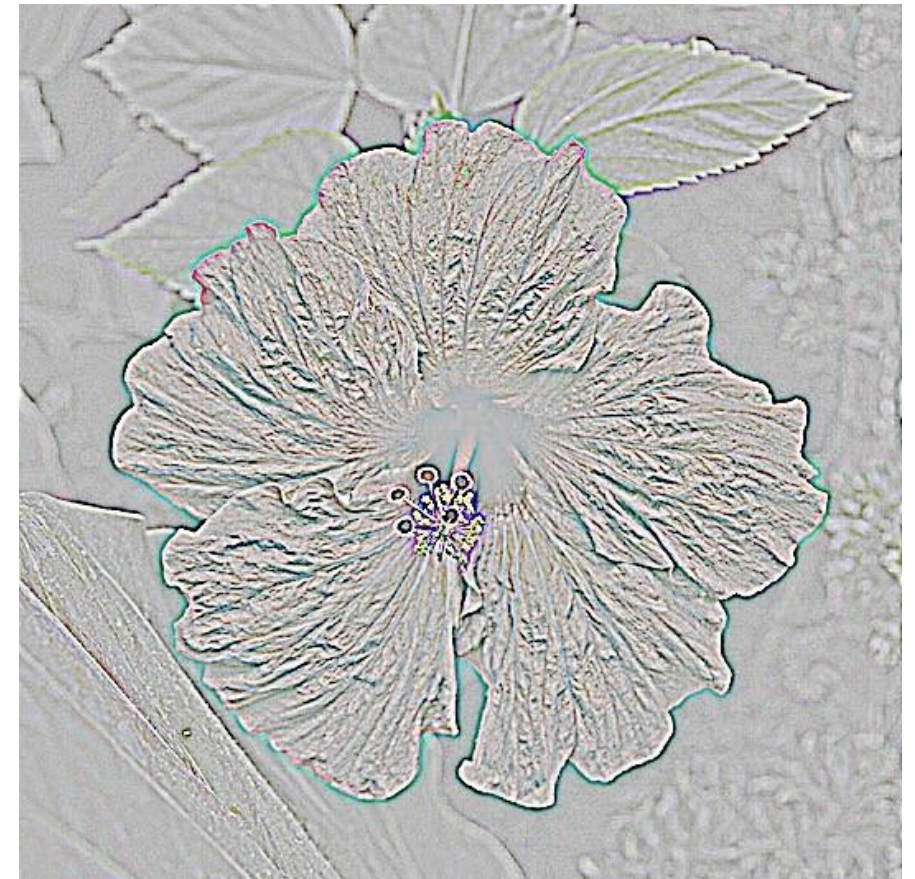
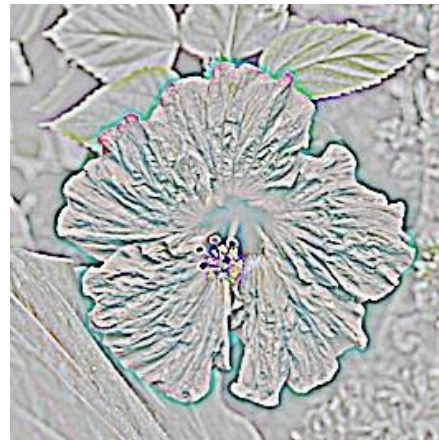
until orig resolution reached

Gaussian vs Laplacian Pyramid



Shown in opposite
order for space.

Which one takes
more space to store?



Still used extensively



foreground details enhanced, background details reduced



input image



user-provided mask

Other types of pyramids

Steerable pyramid: At each level keep multiple versions, one for each direction.



Wavelets: Huge area in image processing



What are image pyramids used for?

image compression



multi-scale texture mapping

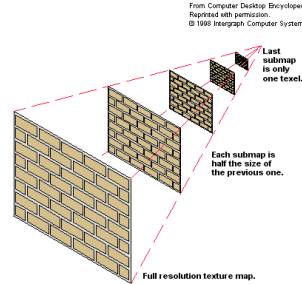
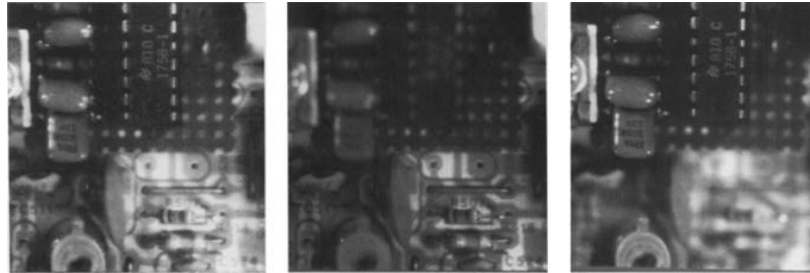


image blending



focal stack compositing



denoising



multi-scale detection



multi-scale registration

